Lab 3

Working with Git History, Stashing, and Reverting Commits

DevOps – Lab 3 Solution

Name: Sadia

GitHub username: Sadia612

Repository: DevOps (Labs/Lab03)

Repository link: https://github.com/Sadia612/DevOps

Environment: Ubuntu server (SSH), GitHub, GitHub Codespaces

Task 1 – Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

A remote edit was made on GitHub and a different local commit was made on Ubuntu. The push was rejected, then resolved first with a merge pull and then with a rebase pull.

Commands used

```
echo "This line was added locally." >> README.md
git add README.md
git commit -m "Local update to README"
git push origin main
git pull --no-rebase origin main
git push -u origin main
nano README.md
git add README.md
git commit -m "Second local update"
git pull --rebase origin main
git push -u origin main
```

Screenshots

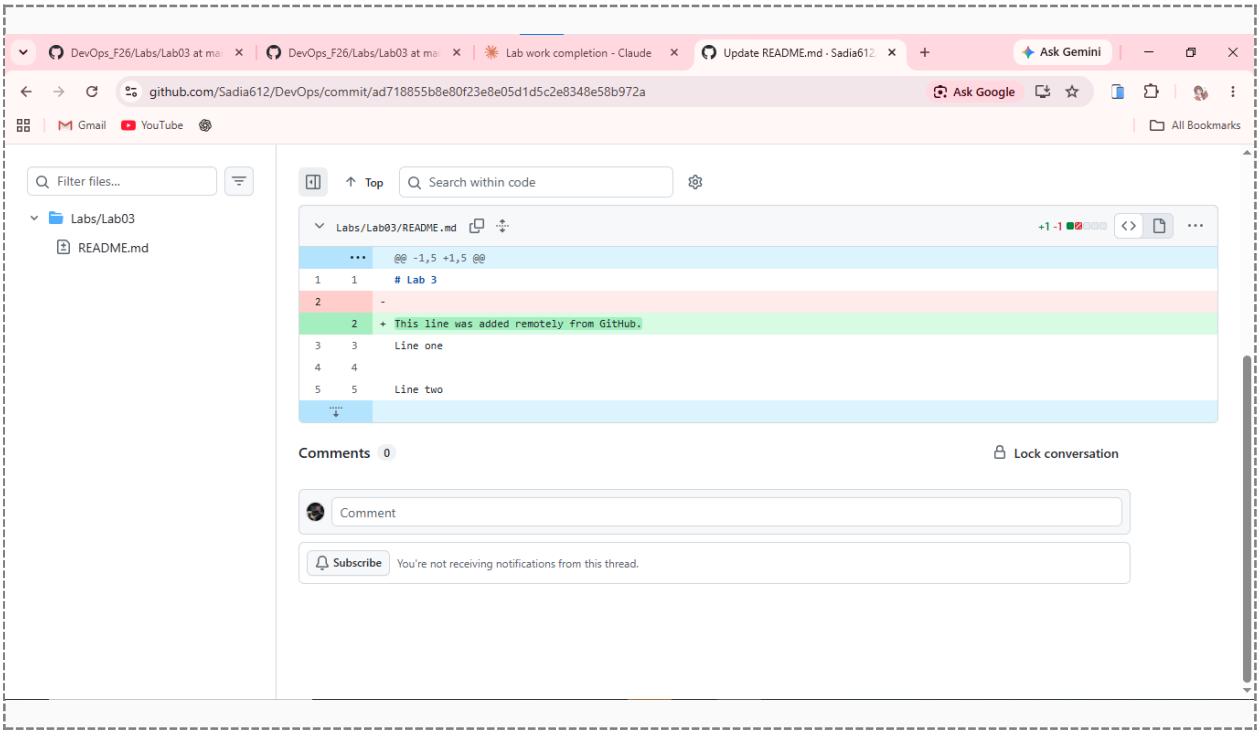


Figure: remote_edit.png

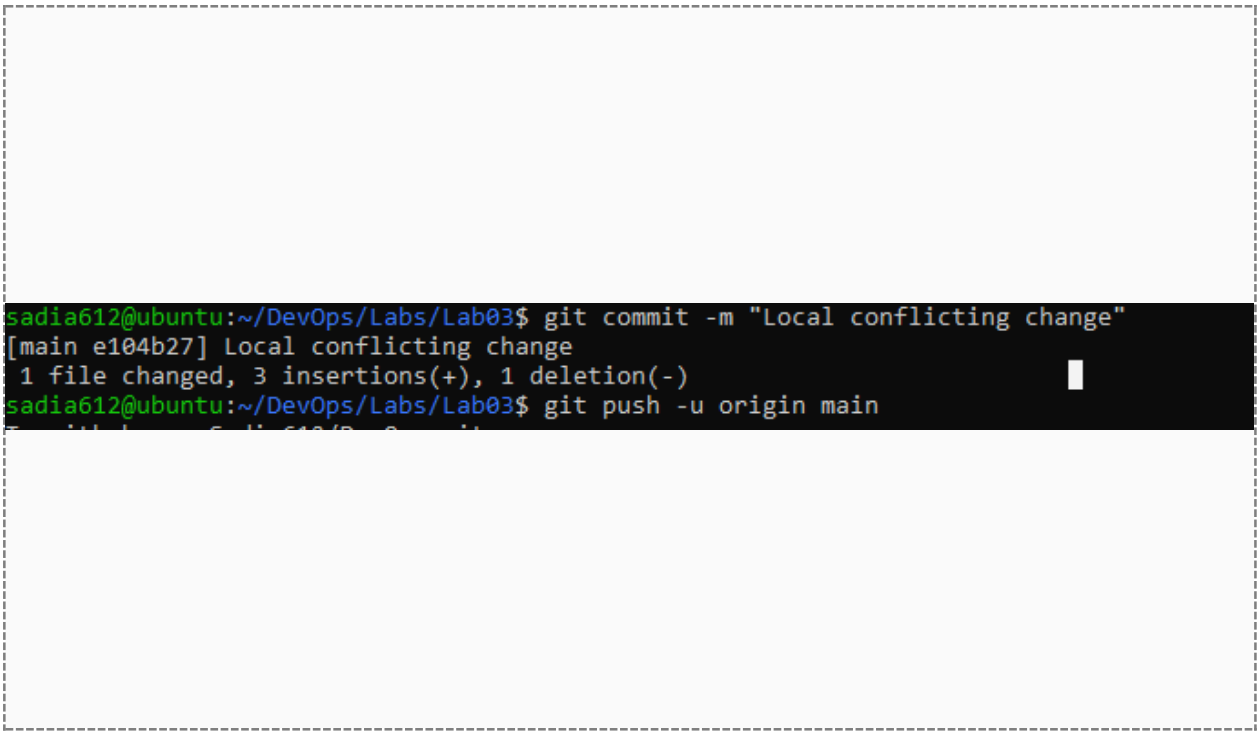


Figure: local_commit.png

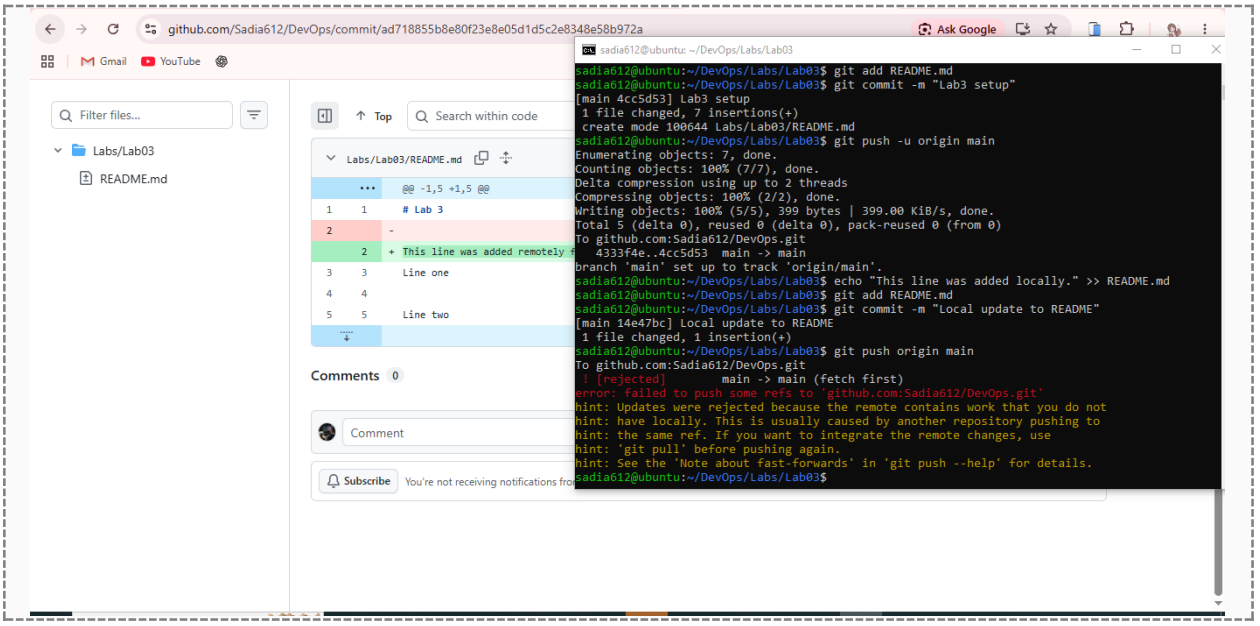


Figure: push_error.png

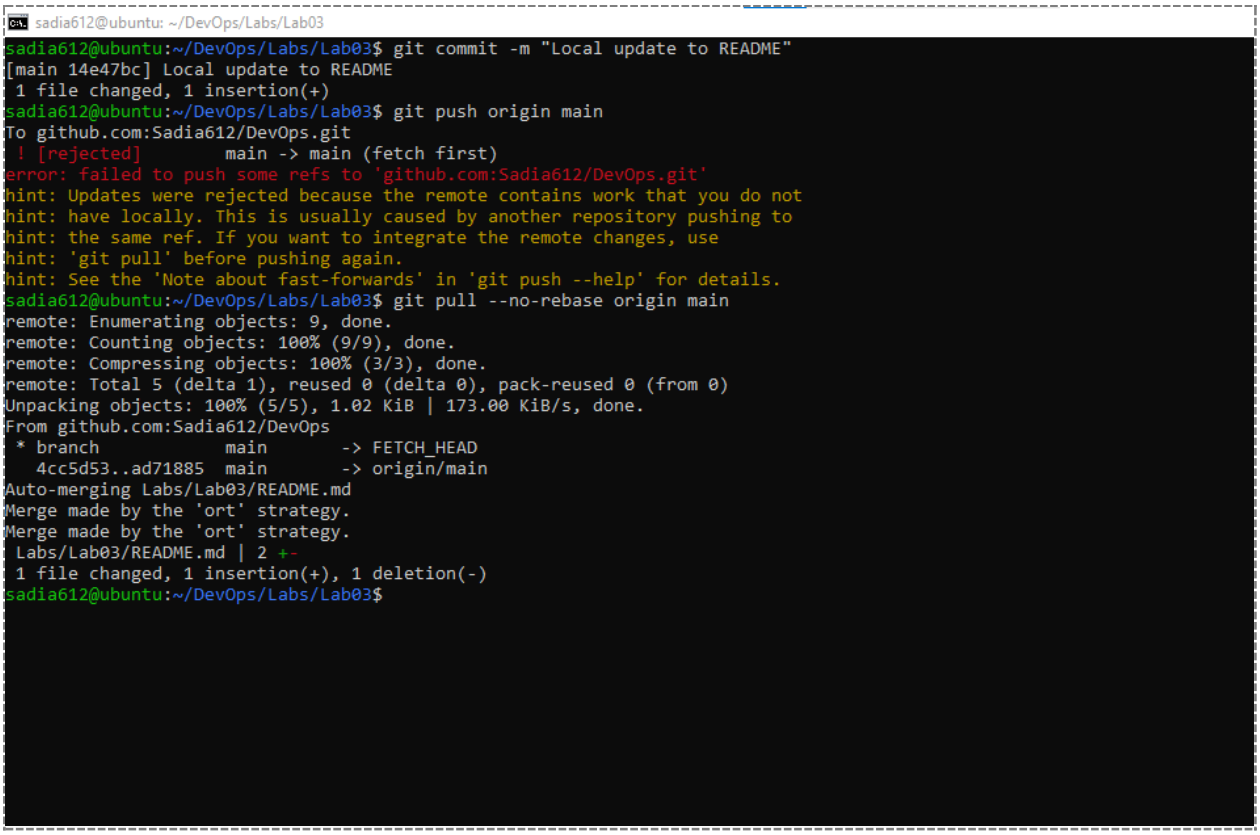


Figure: merge_commit.png

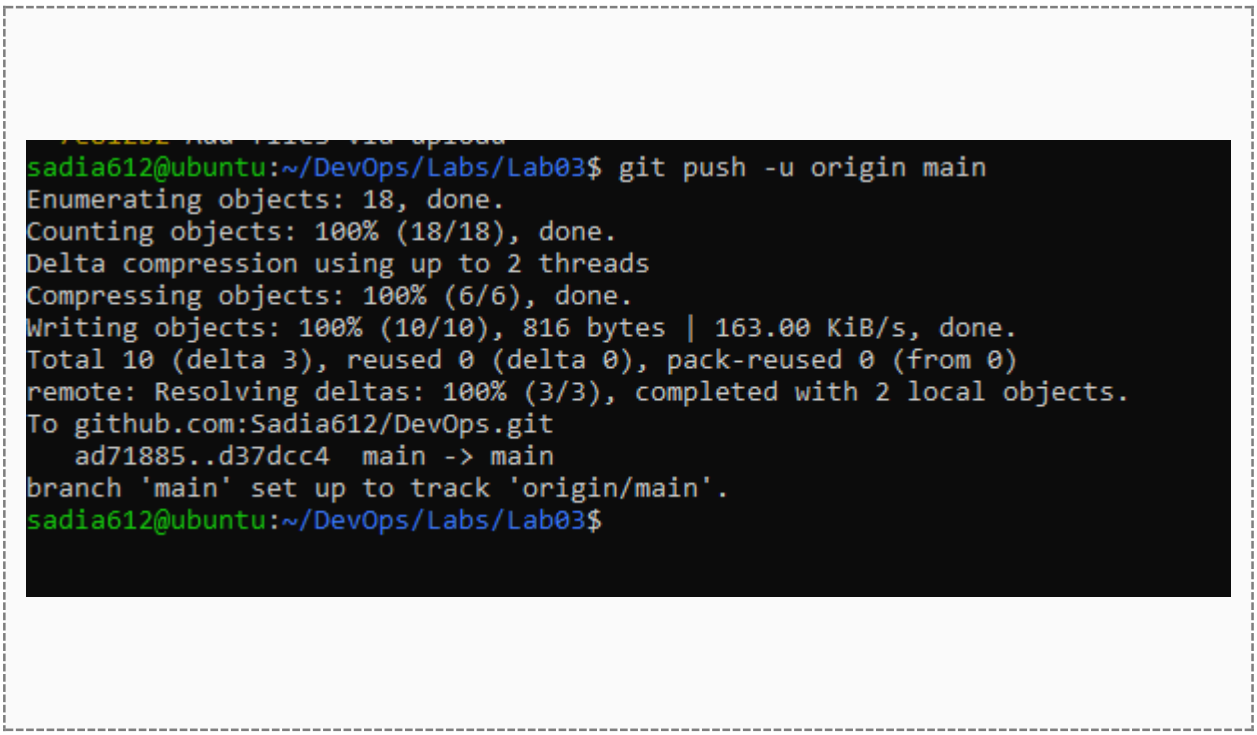


Figure: push_after_merge.png

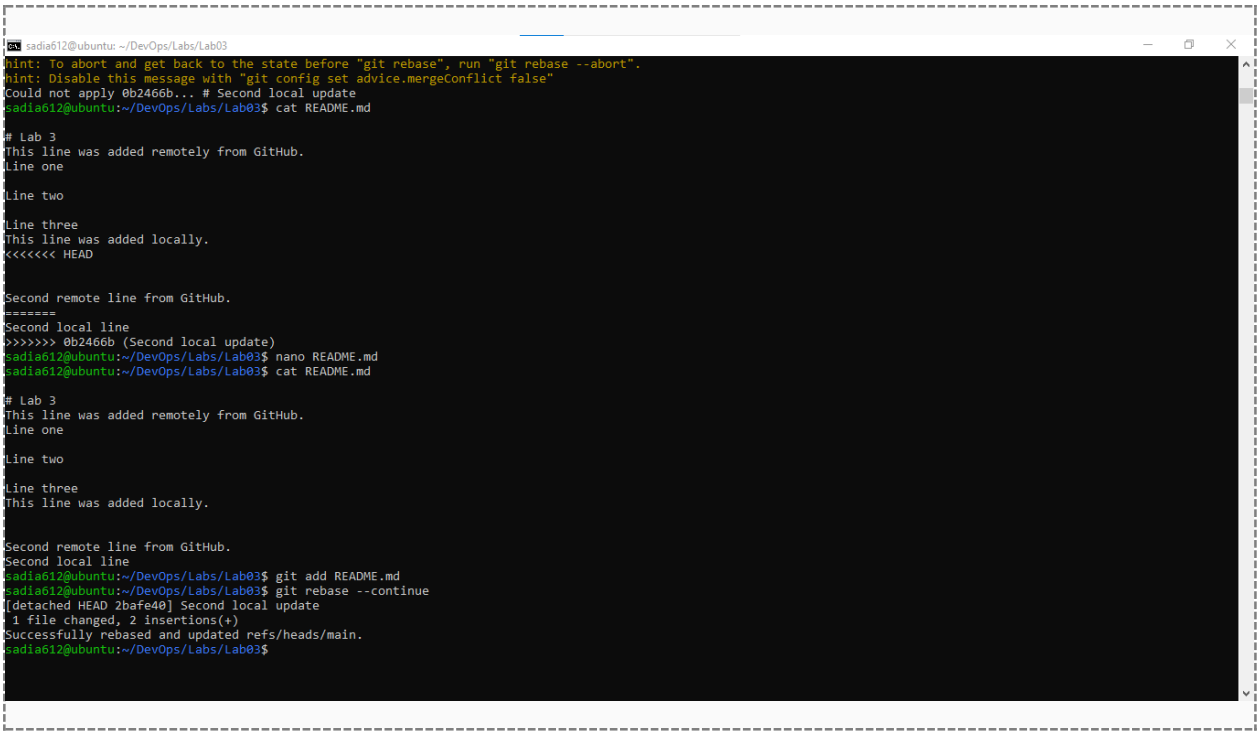


Figure: rebase_pull.png

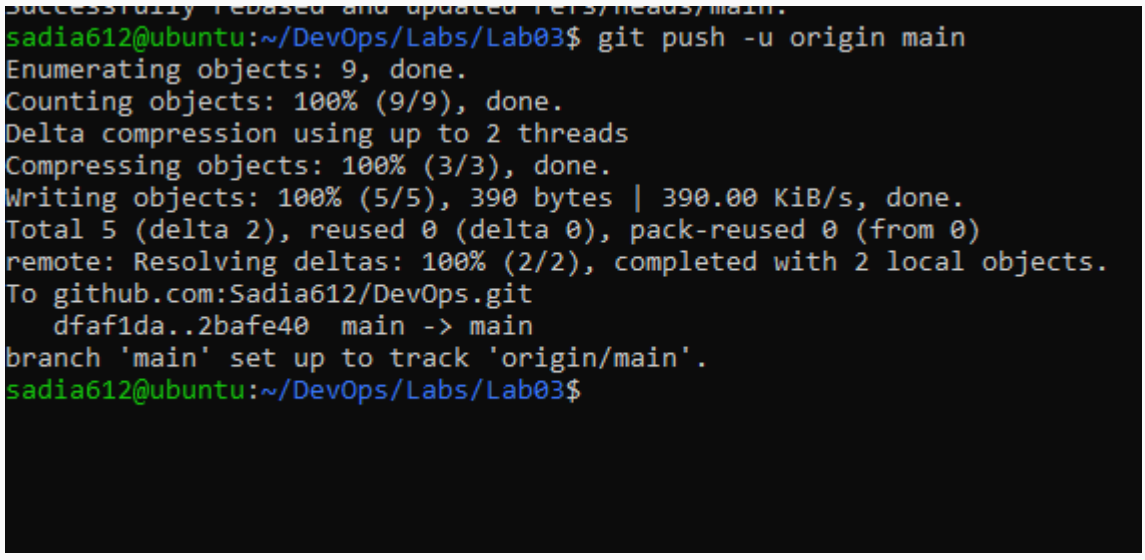


Figure: push_after_rebase.png

Summary

git pull --no-rebase merges remote changes and creates a merge commit. git pull --rebase replays local commits on top of the remote branch, keeping history linear.

Task 2 – Creating and Resolving Merge Conflicts Manually

The same line of README.md was changed differently on GitHub and locally. git pull --rebase paused with a conflict, which was resolved manually by editing the conflict markers.

Commands used

```
git add README.md
git commit -m "Local conflicting change"
git push -u origin main
git pull --rebase origin main
nano README.md
cat README.md
git add README.md
git rebase --continue
git push -u origin main
```

Screenshots

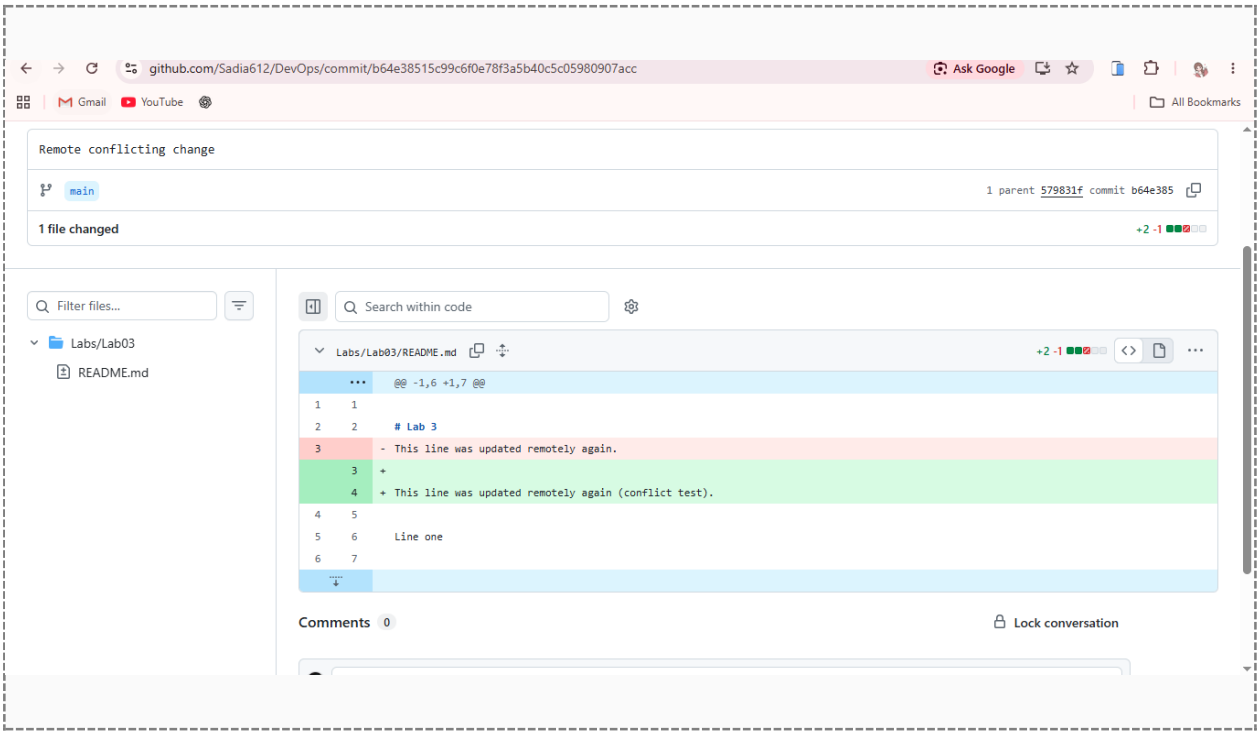


Figure: remote_conflict_edit.png

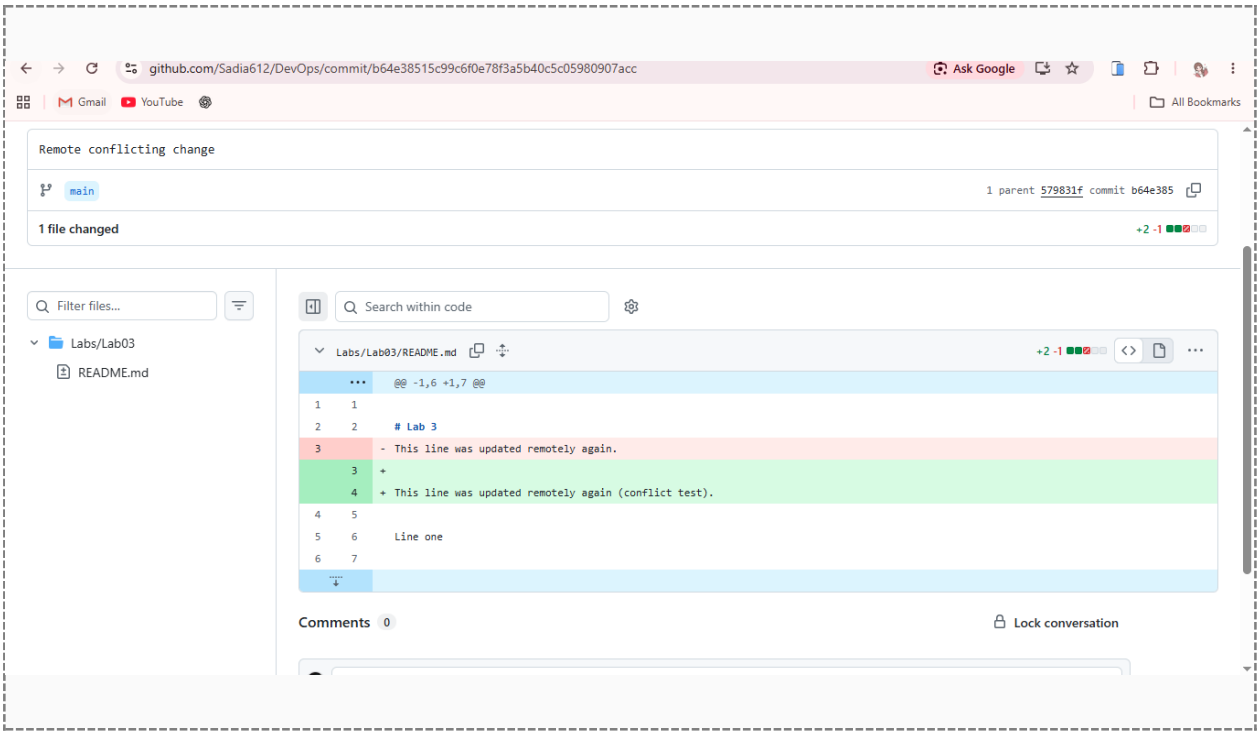


Figure: local_conflict_edit.png

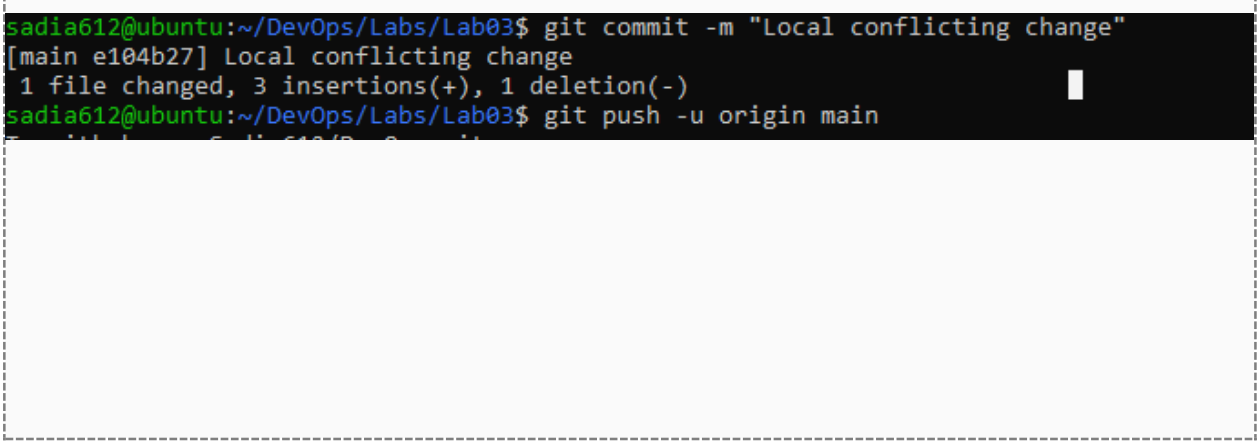


Figure: local_conflict_commit.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git push -u origin main
To github.com:Sadia612/DevOps.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:Sadia612/DevOps.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```



Figure: conflict_push_error.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git pull --rebase origin main
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 15 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (15/15), 3.01 KiB | 128.00 KiB/s, done.
From github.com:Sadia612/DevOps
 * branch            main       -> FETCH_HEAD
   2bafe40..b64e385  main       -> origin/main
Auto-merging Labs/Lab03/README.md
CONFLICT (content): Merge conflict in Labs/Lab03/README.md
error: could not apply e104b27... Local conflicting change
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply e104b27... # Local conflicting change
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

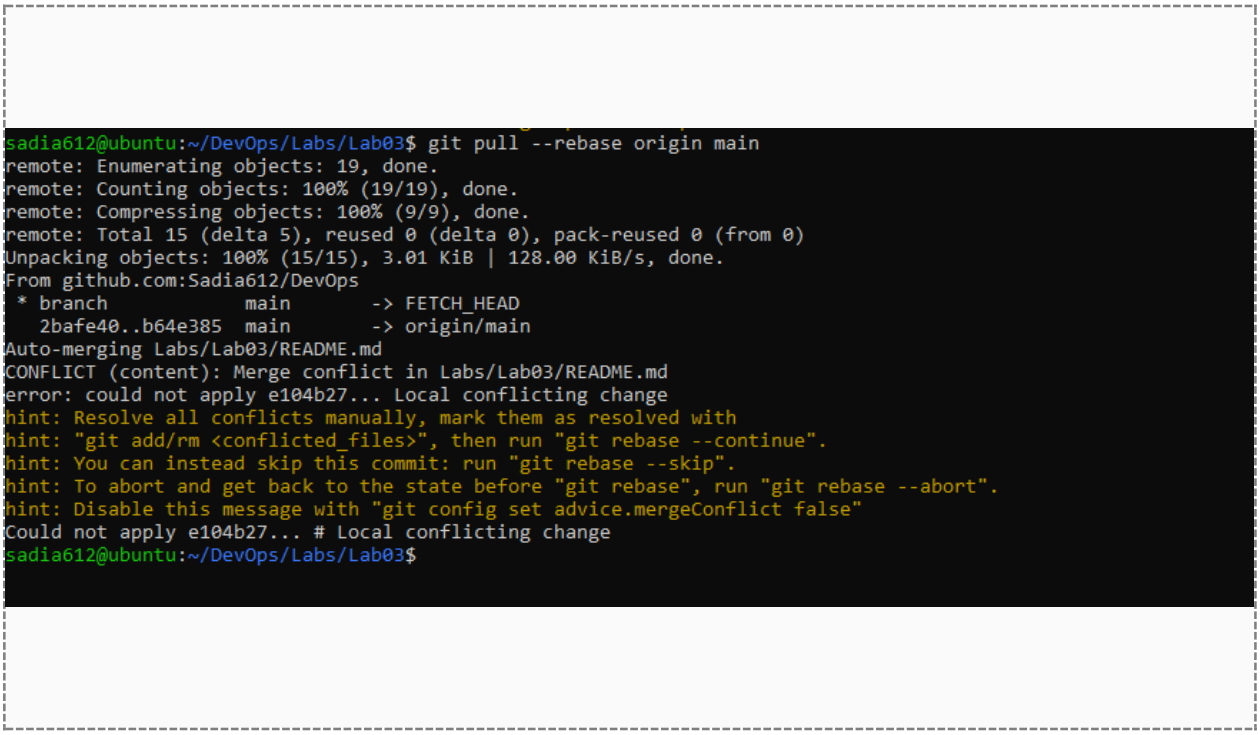


Figure: conflict_message.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ cat README.md

# Lab 3

This line was updated remotely again (conflict test).
This line was updated locally at the same time

Line one

Line two

Line three
This line was added locally.


Second remote line from GitHub.
Second local line
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: resolved_readme.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git rebase --continue
[detached HEAD 70be5e4] Local conflicting change
 1 file changed, 1 insertion(+)
Successfully rebased and updated refs/heads/main.
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: rebase_continue.png

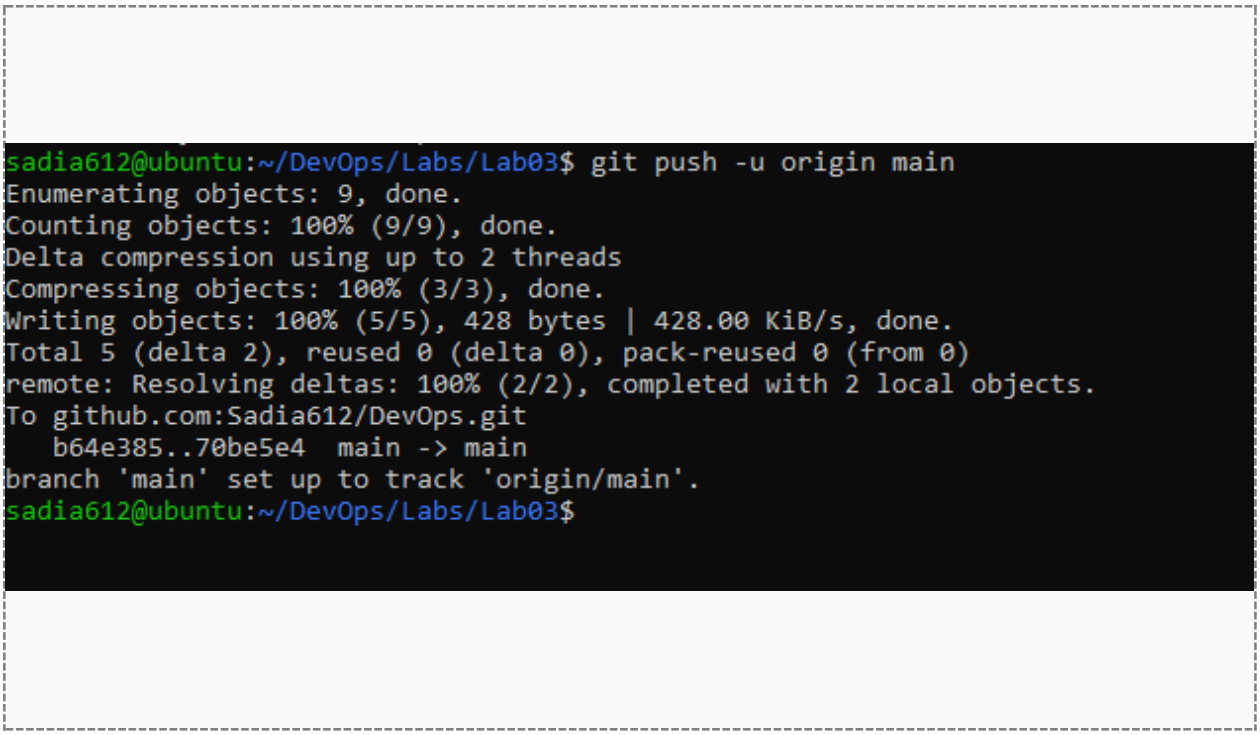


Figure: push_after_resolve.png

Summary

When two people edit the same line, Git stops and asks for manual resolution. After removing the markers and keeping the correct content, continue the rebase and push.

Task 3 – Managing Ignored Files with .gitignore and Removing Tracked Files

A textfiles directory with three files was committed, then ignored with .gitignore, and finally removed from tracking with git rm --cached while remaining on disk.

Commands used

```
mkdir textfiles
echo "File A content" > textfiles/a.txt
echo "File B content" > textfiles/b.txt
echo "File C content" > textfiles/c.txt
git add textfiles
git commit -m "Added textfiles directory with three files"
git push origin main
echo "textfiles/*" > .gitignore
git add .gitignore
git commit -m "Added .gitignore to ignore textfiles directory"
git push origin main
git rm -r --cached textfiles
git commit -m "Removed tracked textfiles directory"
git push origin main
```

Screenshots

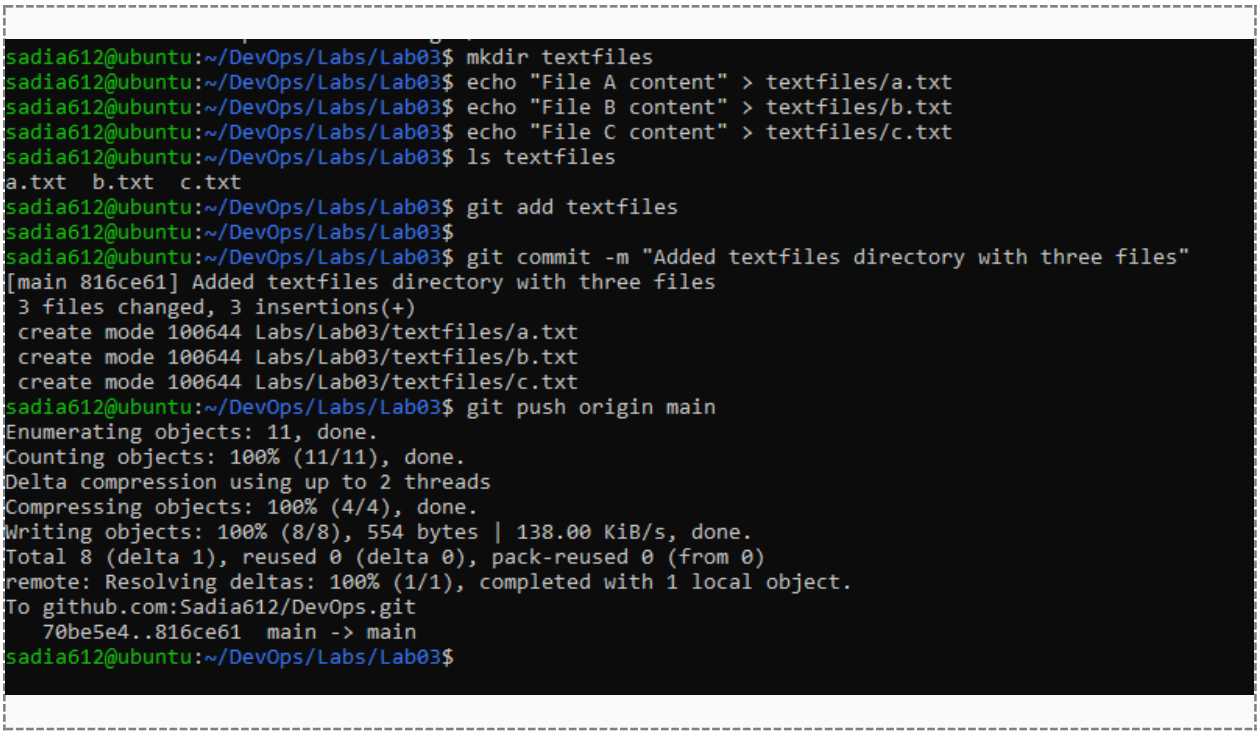


Figure: push_textfiles.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (8/8), 554 bytes | 138.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Sadia612/DevOps.git
   70be5e4..816ce61  main -> main
sadia612@ubuntu:~/DevOps/Labs/Lab03$ echo "textfiles/*" > .gitignore
sadia612@ubuntu:~/DevOps/Labs/Lab03$ cat .gitignore
textfiles/*
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git add .gitignore
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Added .gitignore to ignore textfiles directory"
[main cf23406] Added .gitignore to ignore textfiles directory
 1 file changed, 1 insertion(+)
 create mode 100644 Labs/Lab03/.gitignore
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 440 bytes | 440.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Sadia612/DevOps.git
   816ce61..cf23406  main -> main
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: gitignore_push.png


```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (8/8), 554 bytes | 138.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Sadia612/DevOps.git
   70be5e4..816ce61  main -> main
sadia612@ubuntu:~/DevOps/Labs/Lab03$ echo "textfiles/*" > .gitignore
sadia612@ubuntu:~/DevOps/Labs/Lab03$ cat .gitignore
textfiles/*
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git add .gitignore
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Added .gitignore to ignore textfiles directory"
[main cf23406] Added .gitignore to ignore textfiles directory
 1 file changed, 1 insertion(+)
 create mode 100644 Labs/Lab03/.gitignore
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 440 bytes | 440.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Sadia612/DevOps.git
   816ce61..cf23406  main -> main
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```


Figure: repo_still_has_textfiles.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git rm -r --cached textfiles
rm 'Labs/Lab03/textfiles/a.txt'
rm 'Labs/Lab03/textfiles/b.txt'
rm 'Labs/Lab03/textfiles/c.txt'
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git add .
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Removed tracked textfiles directory"
[main 4dae3c2] Removed tracked textfiles directory
 3 files changed, 3 deletions(-)
 delete mode 100644 Labs/Lab03/textfiles/a.txt
 delete mode 100644 Labs/Lab03/textfiles/b.txt
 delete mode 100644 Labs/Lab03/textfiles/c.txt
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 327 bytes | 327.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:Sadia612/DevOps.git
   cf23406..4dae3c2  main -> main
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: rm_cached_push.png

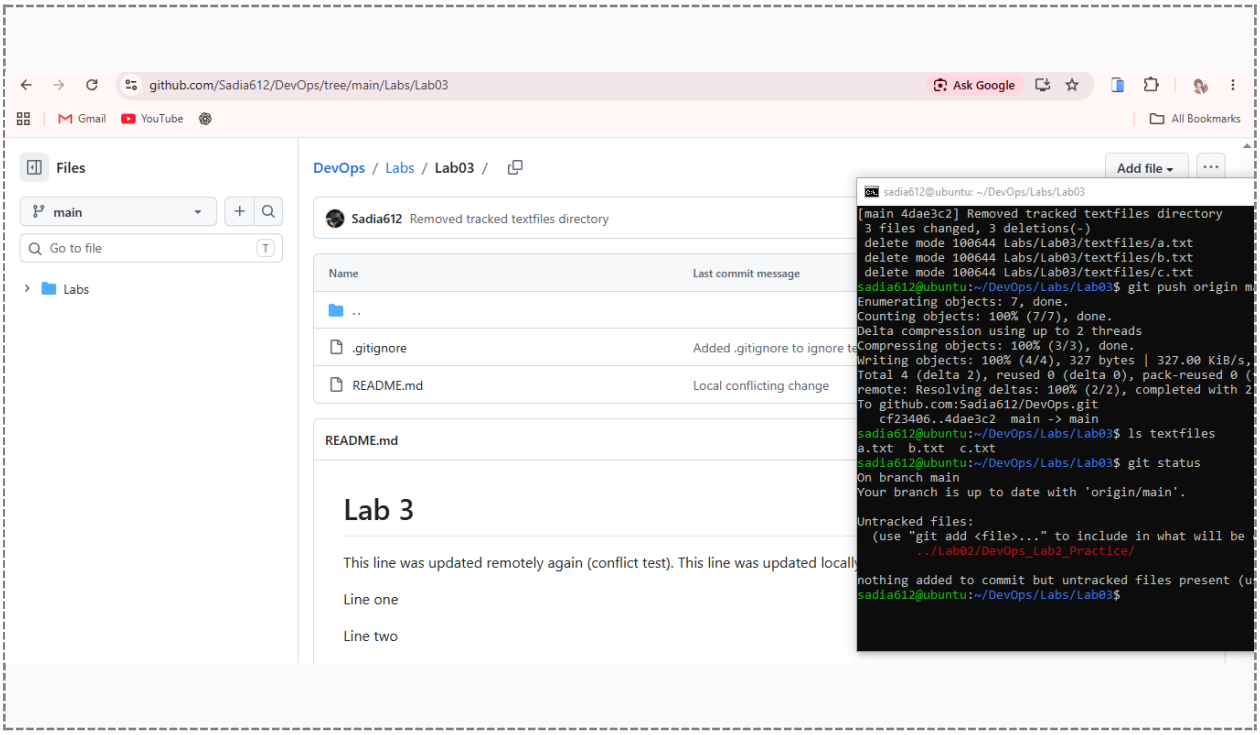


Figure: repo_textfiles_removed.png

Summary

.gitignore does not untrack files that are already committed. Use git rm --cached to stop tracking them while keeping them locally.

Task 4 – Create Temporary Changes and Use git stash

Uncommitted changes blocked a branch switch. They were saved with git stash, the branches were switched, and the changes were restored with git stash pop.

Commands used

```
git checkout -b feature-branch
echo "test line 1" >> README.md
git add README.md
git commit -m "Changes the README.md file"
echo "uncommitted line" >> README.md
git checkout main
git stash
git checkout main
git checkout feature-branch
git status
git stash pop
```

Screenshots

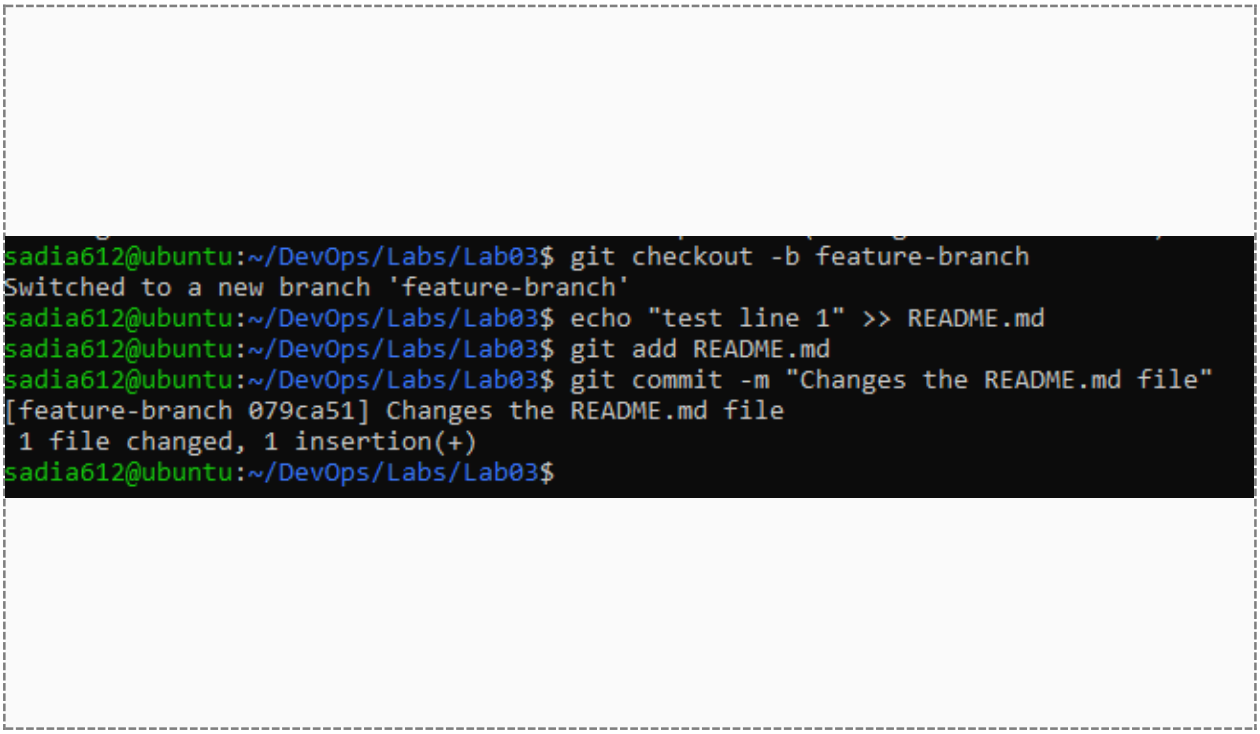


Figure: modified_readme.png

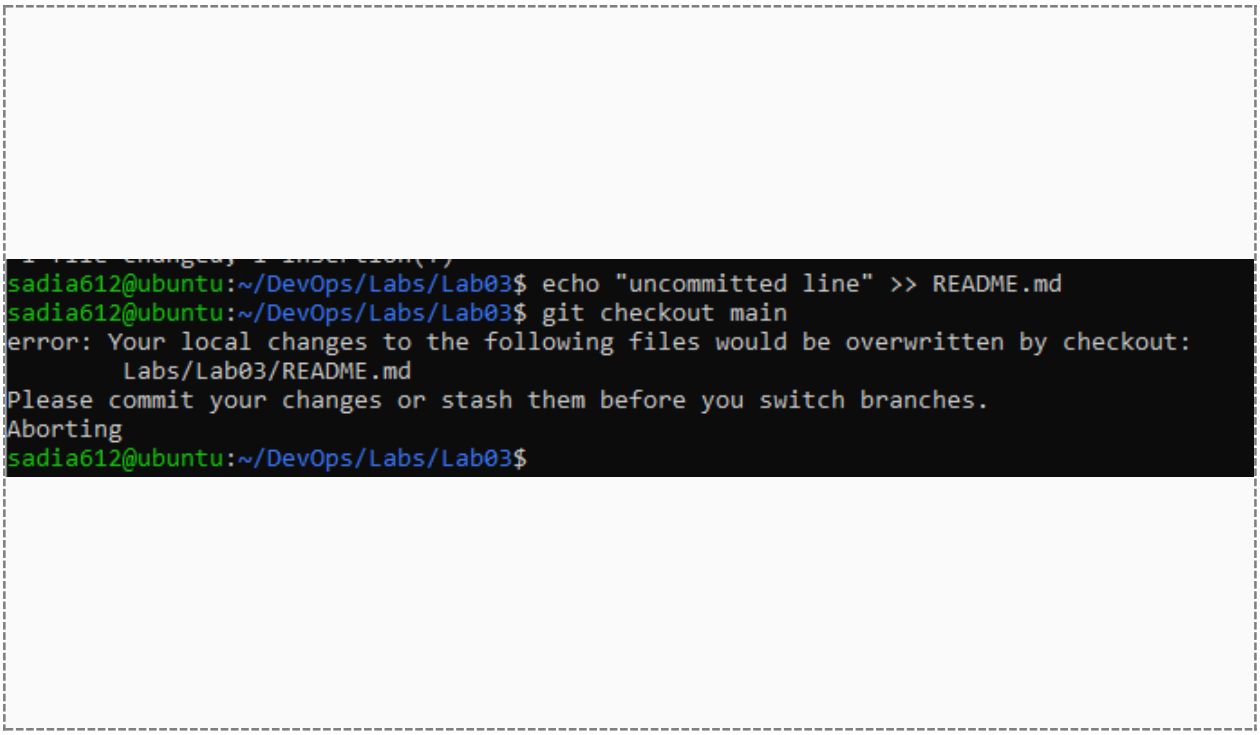


Figure: checkout_error.png



Figure: stash_command.png



Figure: branch_switched.png

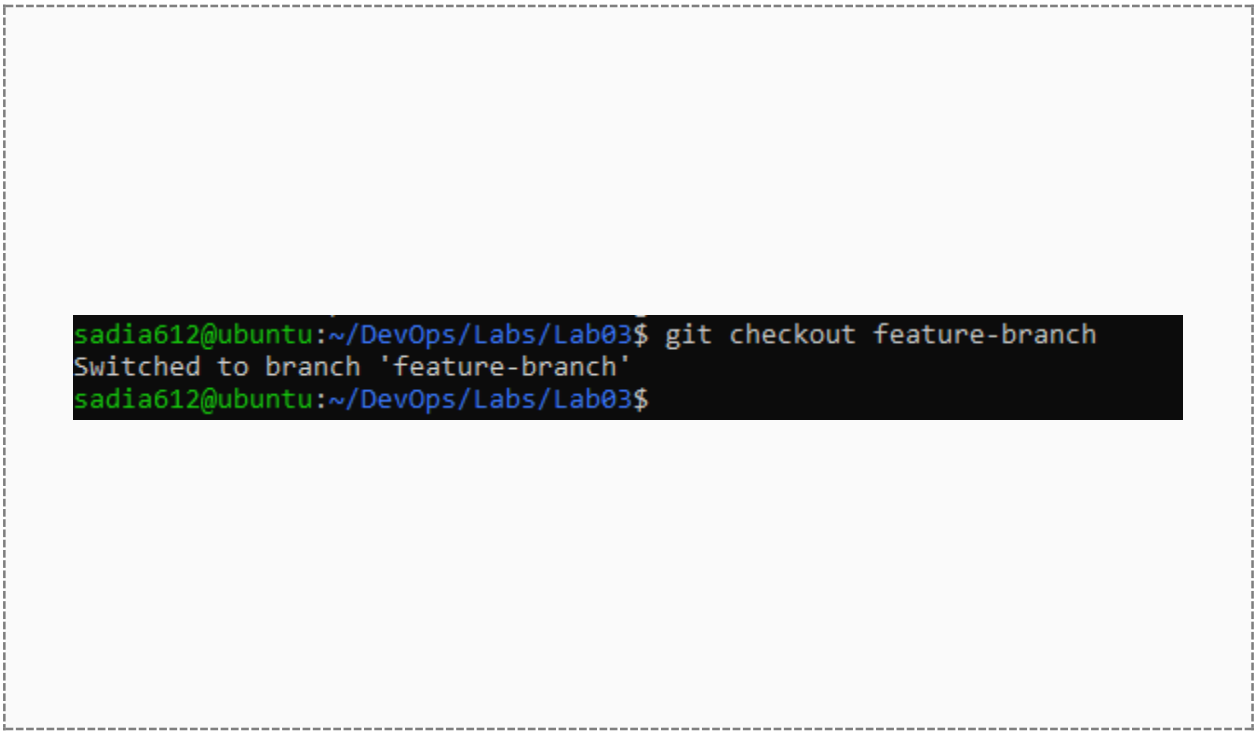


Figure: back_to_feature.png

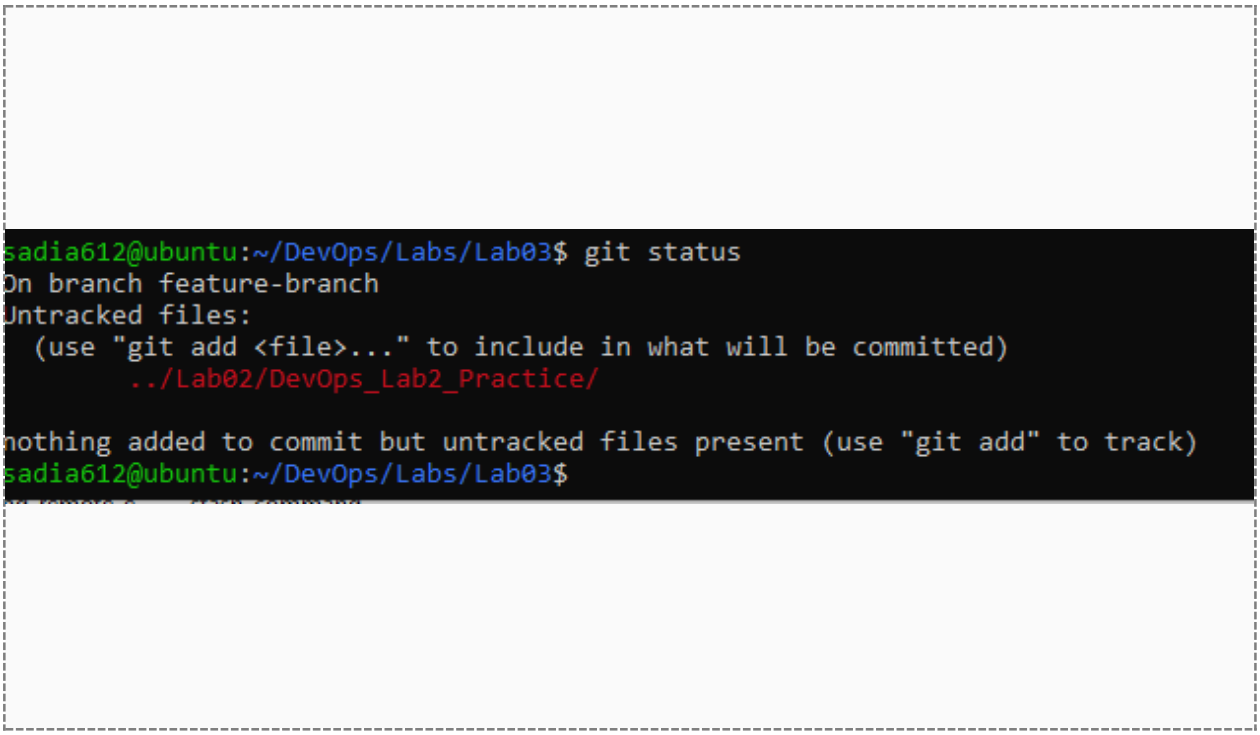


Figure: status_clean.png

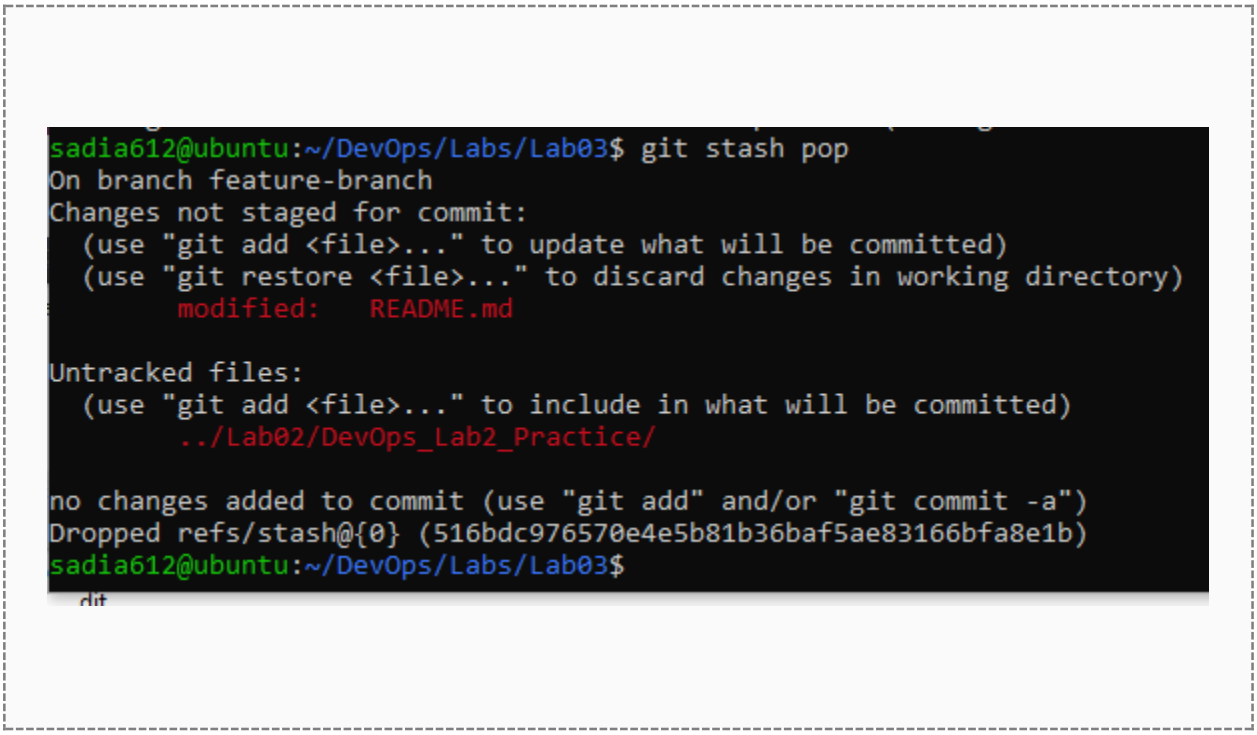


Figure: stash_pop.png

Summary

git stash saves uncommitted work so you can switch branches safely, and git stash pop brings it back.

Task 5 – Checkout a Specific Commit Using git log

The commit history was inspected and an older commit was checked out, putting the repository in a detached HEAD state before returning to main.

Commands used

```
git log --oneline
git checkout 4cc5d53
git checkout main
```

Screenshots

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline
0f4a854 (HEAD -> main, origin/main, origin/HEAD, feature-branch) WIP from feature
079ca51 Changes the README.md file
4dae3c2 Removed tracked textfiles directory
cf23406 Added .gitignore to ignore textfiles directory
816ce61 Added textfiles directory with three files
70be5e4 Local conflicting change
b64e385 Remote conflicting change
579831f Update README.md
82d427a Update README.md
2bafe40 Second local update
dfaf1da Update README.md
bd66d0c Update README.md
d37dcc4 Merge branch 'main' of github.com:Sadia612/DevOps
14e47bc Local update to README
ad71885 Update README.md
4cc5d53 Lab3 setup
4333f4e Add files via upload
a3c3a99 Add files via upload
4306f6a Add files via upload
7e812b2 Add files via upload
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: log_before_checkout.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git checkout 4cc5d53
Note: switching to '4cc5d53'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 4cc5d53 Lab3 setup
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: detached_head.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git checkout 4cc5d53
Note: switching to '4cc5d53'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 4cc5d53 Lab3 setup
sadia612@ubuntu:~/DevOps/Labs/Lab03$ cat README.md
# Lab 3

Line one

Line two

Line three
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git checkout main
Previous HEAD position was 4cc5d53 Lab3 setup
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: back_to_main.png

Summary

Checking out a commit hash lets you inspect old versions. In detached HEAD state commits belong to no branch, so always return to main afterwards.

Task 6 – Resetting Commits (Soft vs Hard Reset)

Two commits were created. A soft reset removed the last commit but kept the changes staged; after re-committing, a hard reset removed the commit and its changes from the file.

Commands used

```
echo "test line A" >> README.md
git add README.md
git commit -m "Added test line"
echo "test line B" >> README.md
git add README.md
git commit -m "Second test commit"
git log --oneline
git reset --soft HEAD~1
git log --oneline
git status
git commit -m "Second Test commit"
git reset --hard HEAD~1
git log --oneline
git status
```

Screenshots

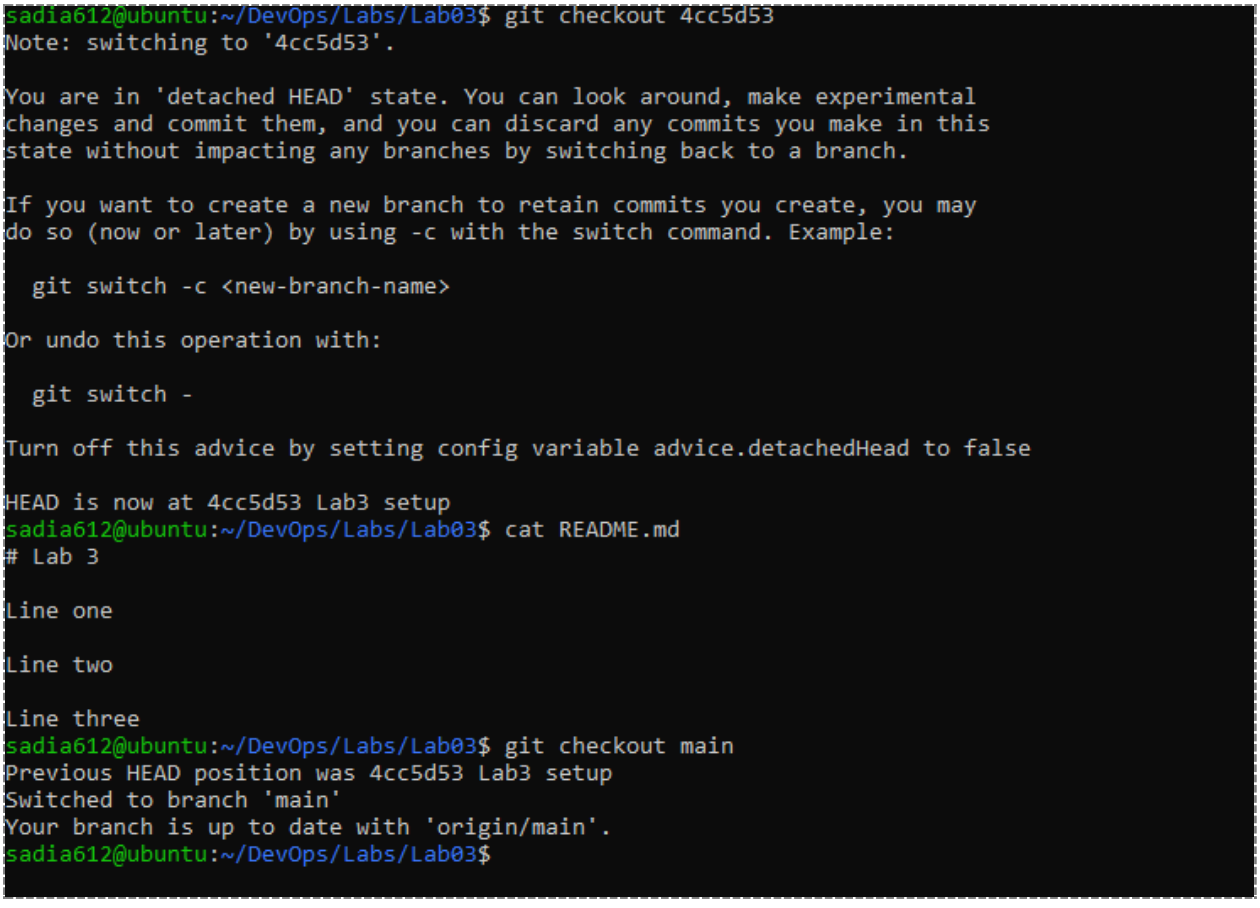


Figure: first_commit.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ echo "test line B" >> README.md
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git add README.md
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Second test commit"
[main ca5be4d] Second test commit
 1 file changed, 1 insertion(+)
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: second_commit.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git log --oneline | head -5
ca5be4d Second test commit
1d8014c Added test line
0f4a854 WIP from feature
079ca51 Changes the README.md file
4dae3c2 Removed tracked textfiles directory
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: log_before_reset.png

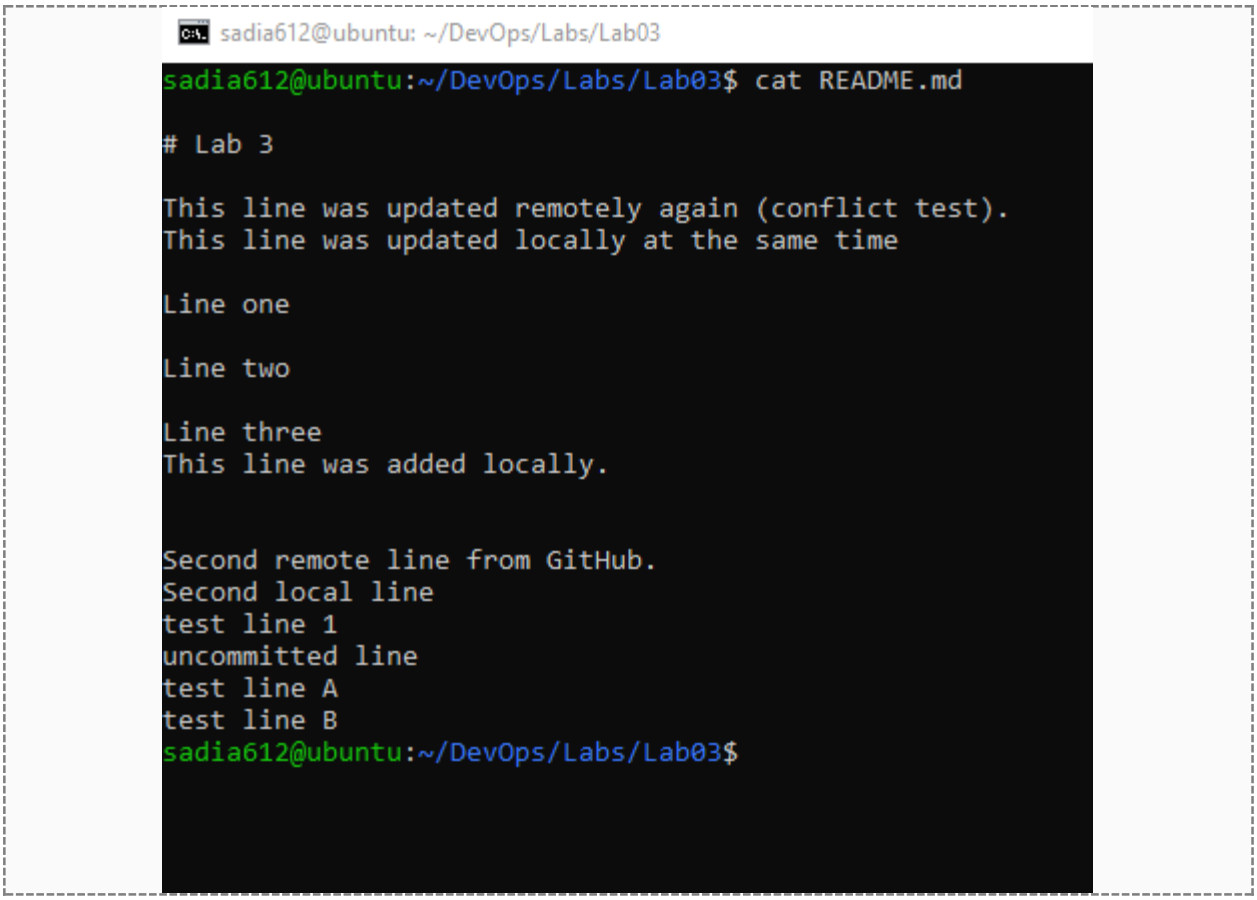


Figure: file_before_reset.png

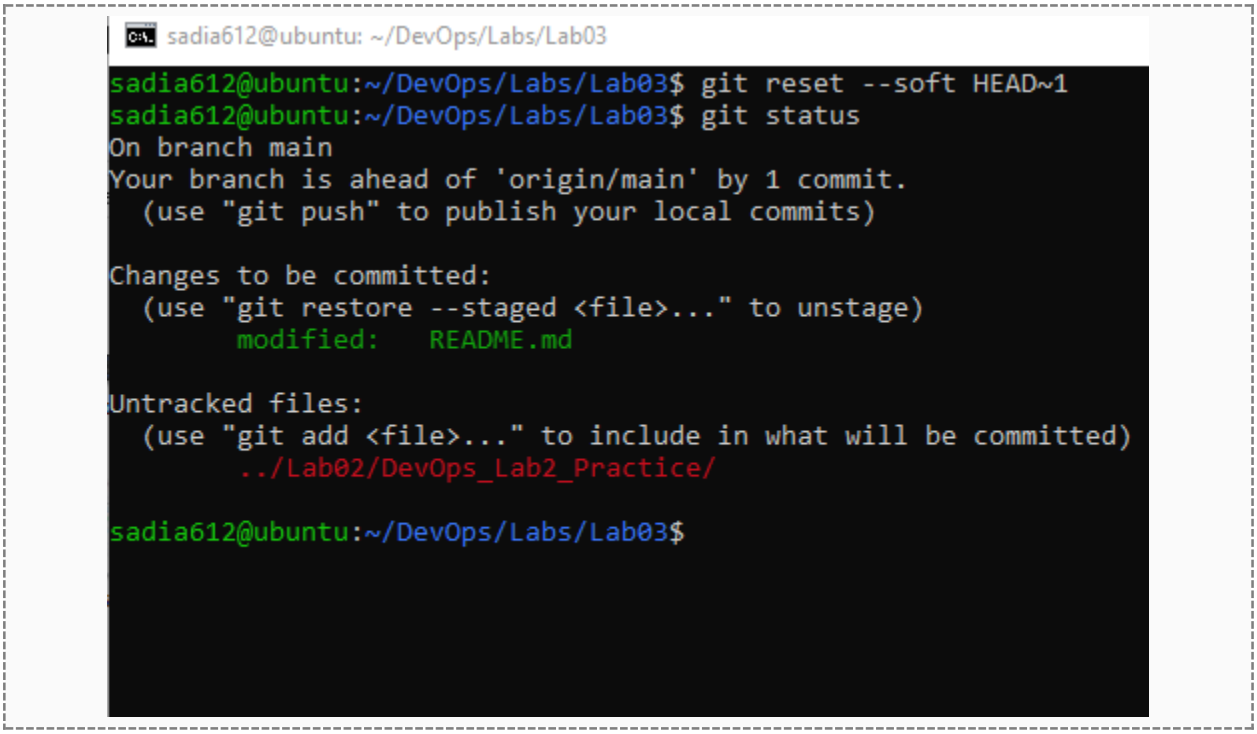


Figure: soft_reset.png

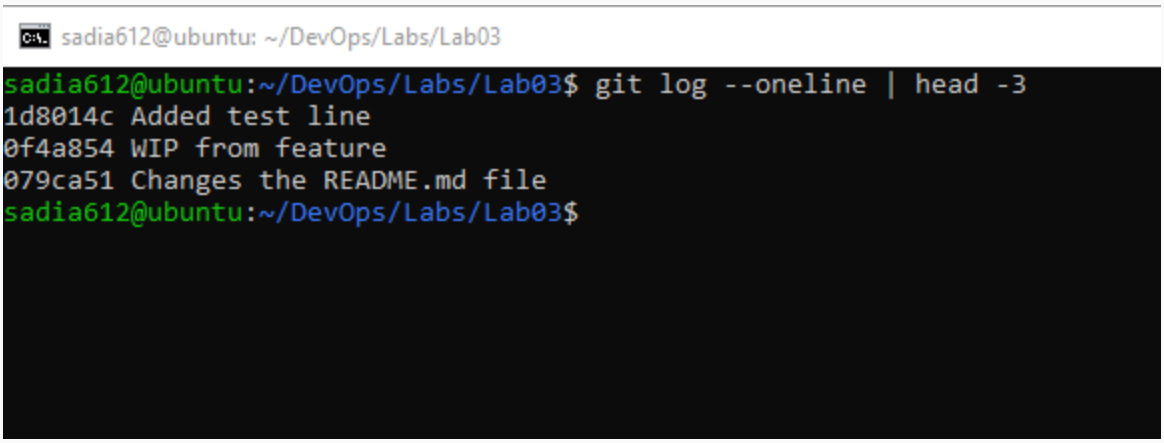


Figure: log_after_soft_reset.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ cat README.md

# Lab 3

This line was updated remotely again (conflict test).
This line was updated locally at the same time

Line one

Line two

Line three
This line was added locally.


Second remote line from GitHub.
Second local line
test line 1
uncommitted line
test line A
test line B
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../Lab02/DevOps_Lab2_Practice/

sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: file_after_soft_reset.png



Figure: recommit_after_soft.png



Figure: hard_reset.png

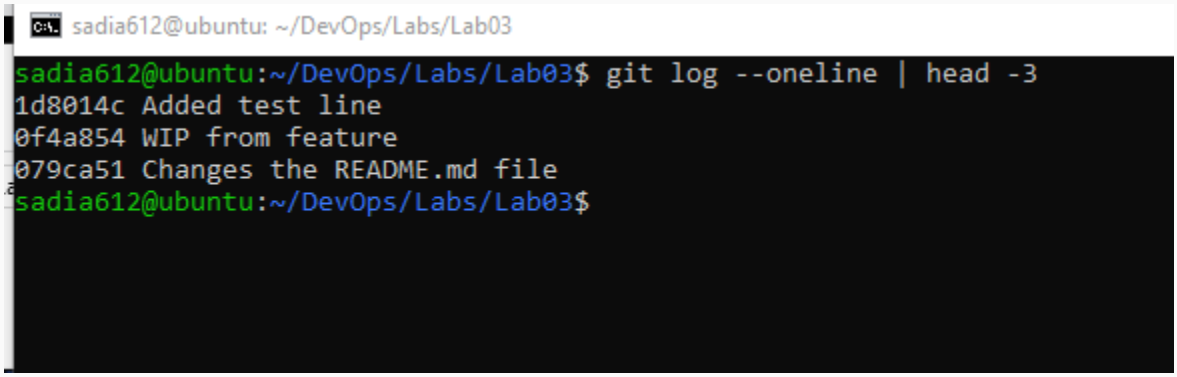


Figure: log_after_hard_reset.png

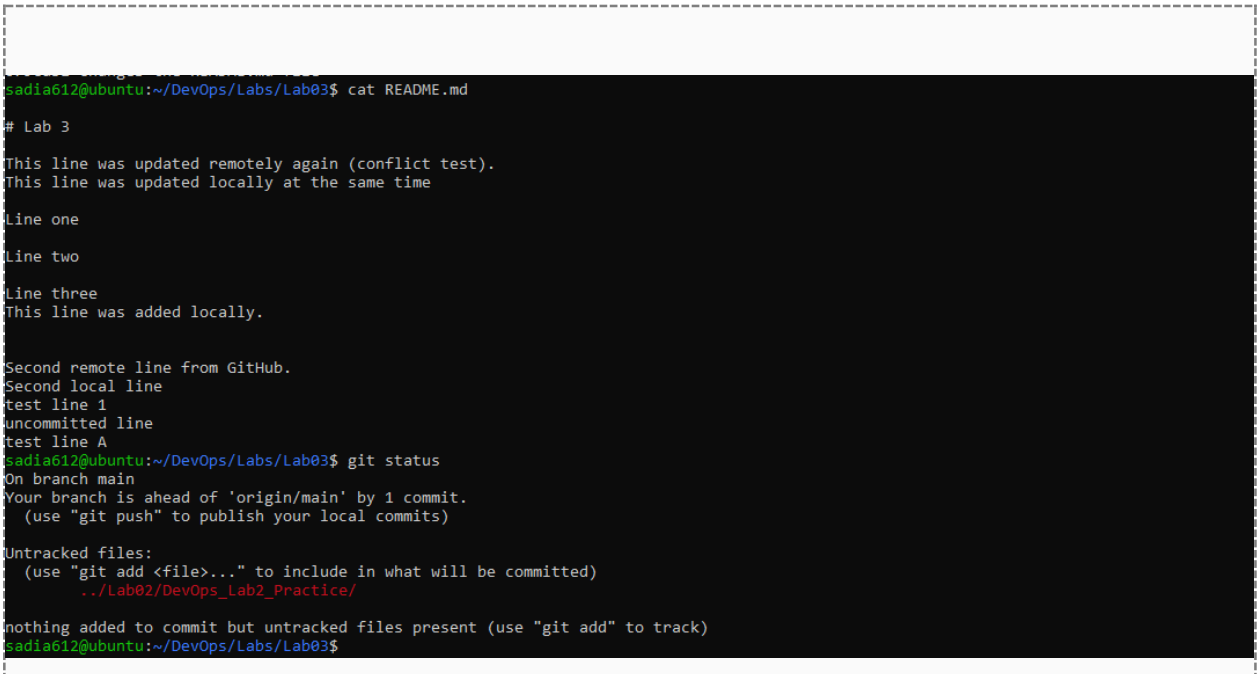


Figure: file_after_hard_reset.png

Summary

Soft reset keeps the changes in the working tree and index. Hard reset discards the commit and the changes.

Task 7 – Amending the Last Commit

A commit was created and then amended to include a second file without creating a new commit.

Commands used

```
echo "fix" >> README.md
git add README.md
git commit -m "Fix log message"
echo "another file" > other.txt
git add other.txt
git commit --amend
git show --stat HEAD
```

Screenshots

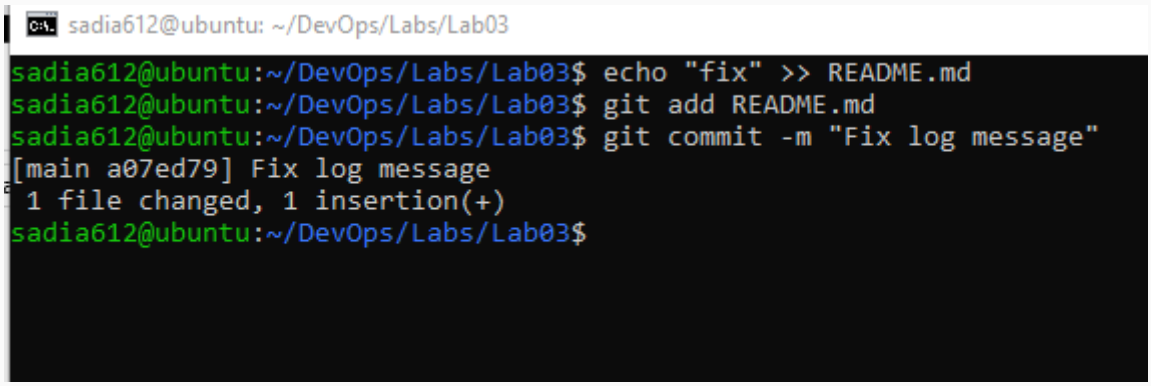


Figure: first_amend_commit.png

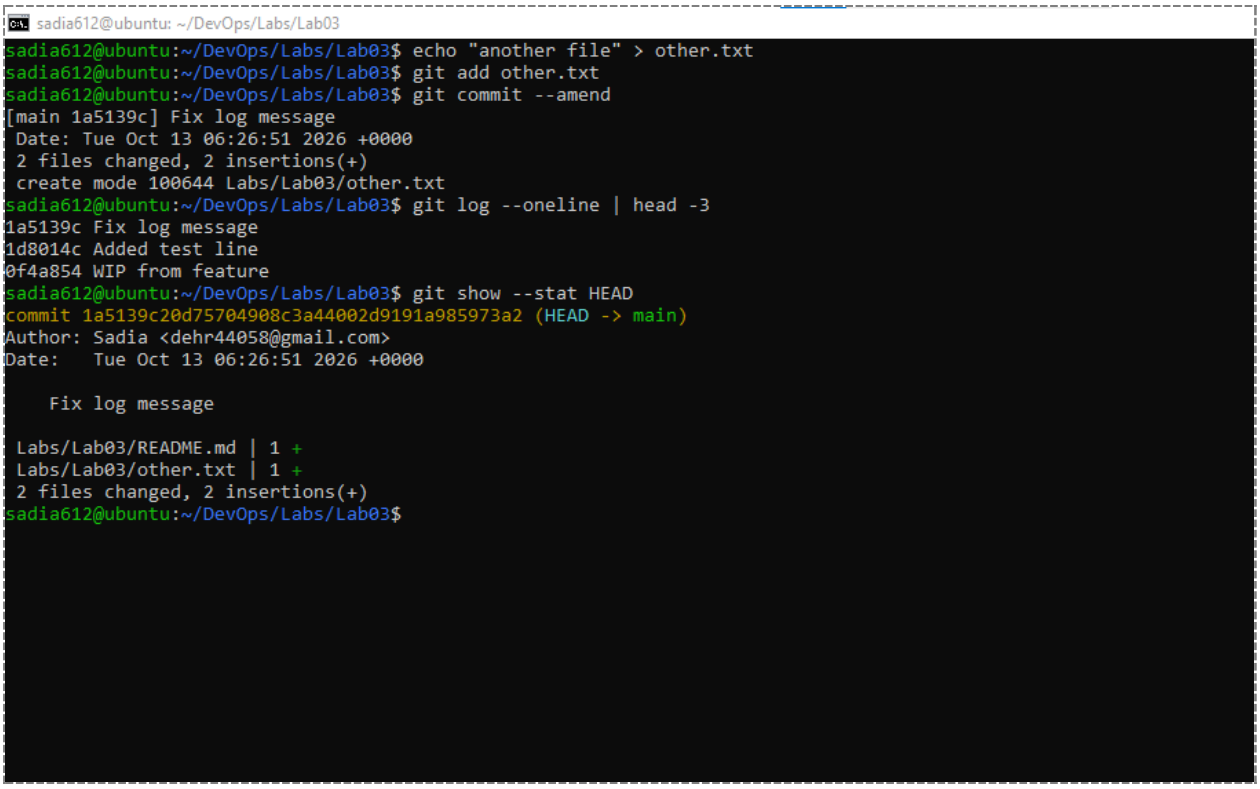


Figure: amend_commit.png

Summary

git commit --amend modifies the last commit. After pushing, amending requires a force push.

Task 8 – Reverting a Commit (Safe Undo on Remote Branch)

A temporary file was committed and pushed, then undone with git revert, which created a new commit instead of rewriting history.

Commands used

```
echo "temporary text" >> temp.txt
git add temp.txt
git commit -m "Added temporary text file"
git push origin main
git log --oneline
git revert cf6ad92
git push origin main
```

Screenshots

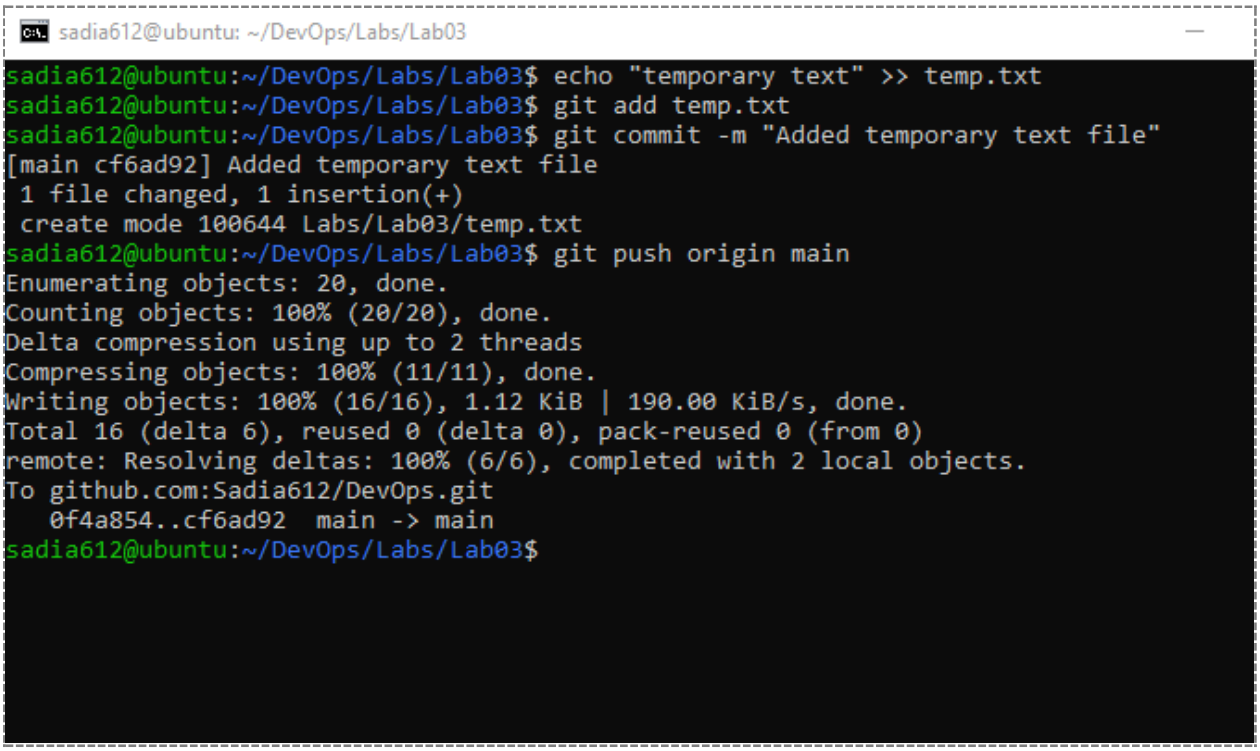


Figure: commit_temp_file.png

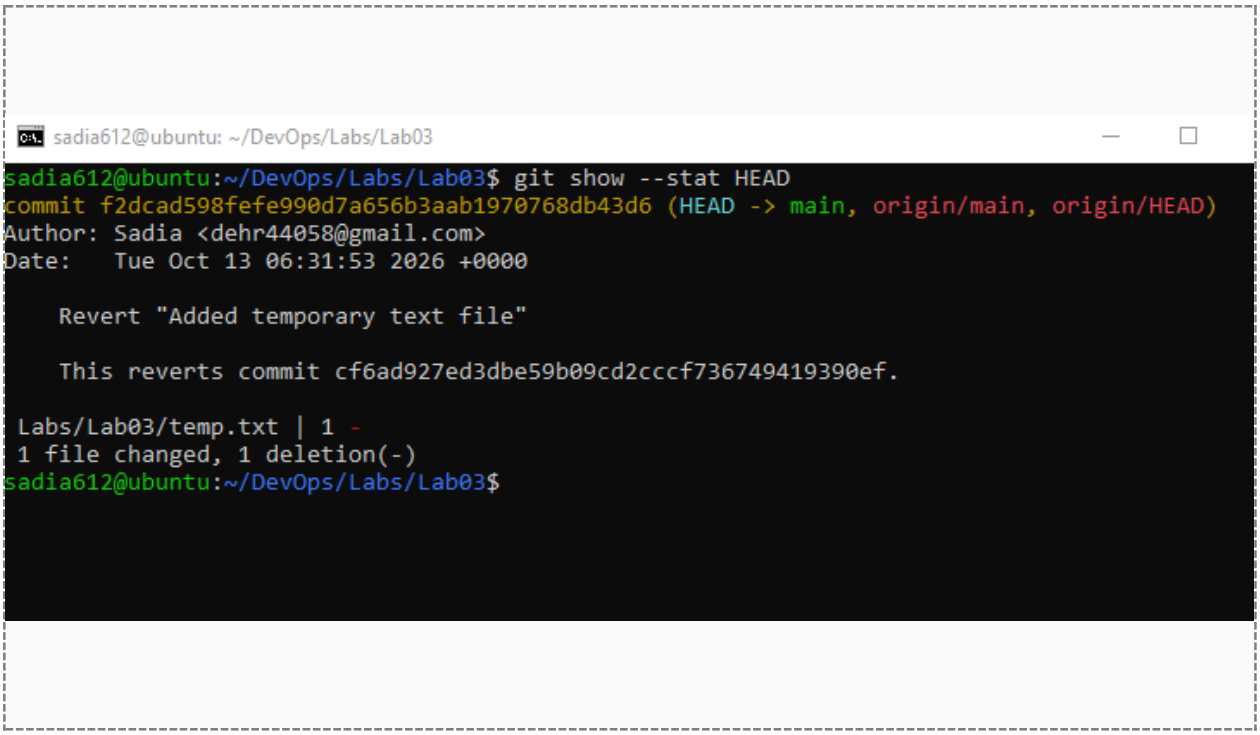


Figure: revert_commit.png

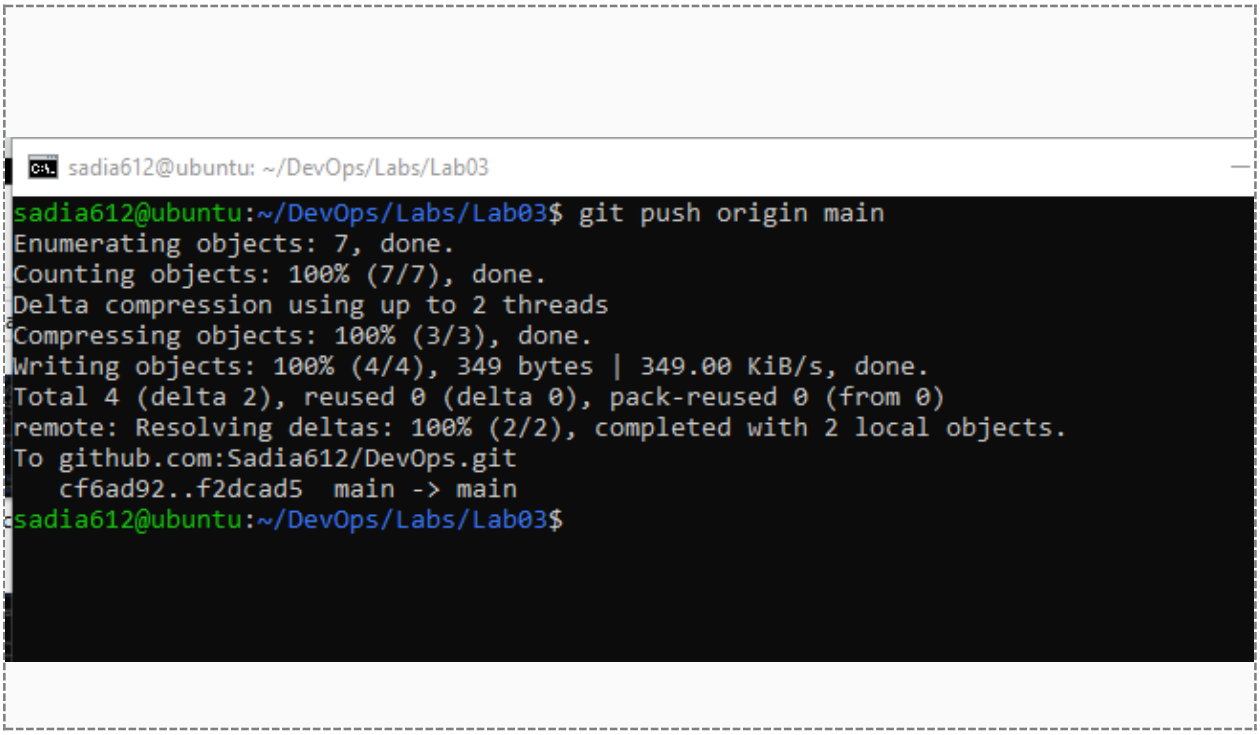


Figure: revert_push.png

Summary

git revert creates a new commit that undoes an earlier one and is safe on shared branches.

Task 9 – Force Push (With Caution)

On a separate branch, a commit was pushed and then removed locally with a hard reset. A normal push was rejected, and a force push overwrote the remote history.

Commands used

```
git checkout -b test-force
echo "force test" >> force.txt
git add force.txt
git commit -m "Force test commit"
git push origin test-force
git reset --hard HEAD~1
git push origin test-force
git push origin test-force --force
```

Screenshots



Figure: new_branch.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ echo "force test" >> force.txt
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git add force.txt
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git commit -m "Force test commit"
[test-force 7fa479c] Force test commit
 1 file changed, 1 insertion(+)
 create mode 100644 Labs/Lab03/force.txt
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: force_commit.png

```
sadia612@ubuntu:~/DevOps/Labs/Lab03$ git push origin test-force
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 447 bytes | 223.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'test-force' on GitHub by visiting:
remote:      https://github.com/Sadia612/DevOps/pull/new/test-force
remote:
To github.com:Sadia612/DevOps.git
 * [new branch]      test-force -> test-force
sadia612@ubuntu:~/DevOps/Labs/Lab03$
```

Figure: push_force_branch.png



Figure: hard_reset_force.png

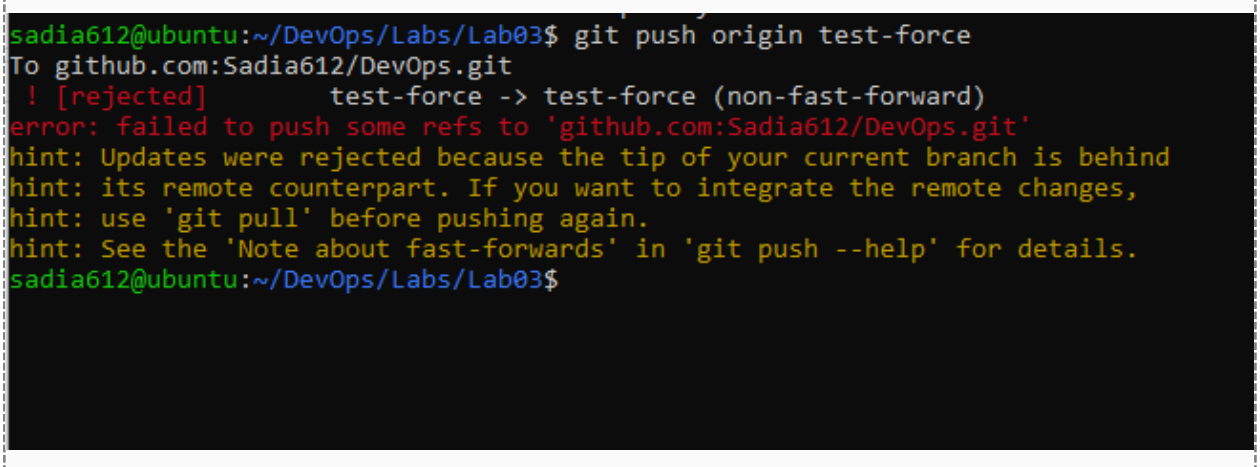


Figure: normal_push.png



Figure: force_push.png

Summary

git push --force rewrites remote history. Use it only on your own non-shared branches.

Task 10 – Running Gitea in GitHub Codespaces via Docker Compose

The Gitea repository was forked and opened in a Codespace. Gitea was run with Docker Compose using compose.yaml (port 3000 published and made public), installed through the web interface, and a repository was created.

Commands used

```
docker compose up -d
docker ps
```

File: compose.yaml

```
services:
  server:
    image: gitea/gitea:latest
    container_name: gitea
    restart: always
    ports:
      - "3000:3000"
    volumes:
      - ./gitea-data:/data
```

Screenshots

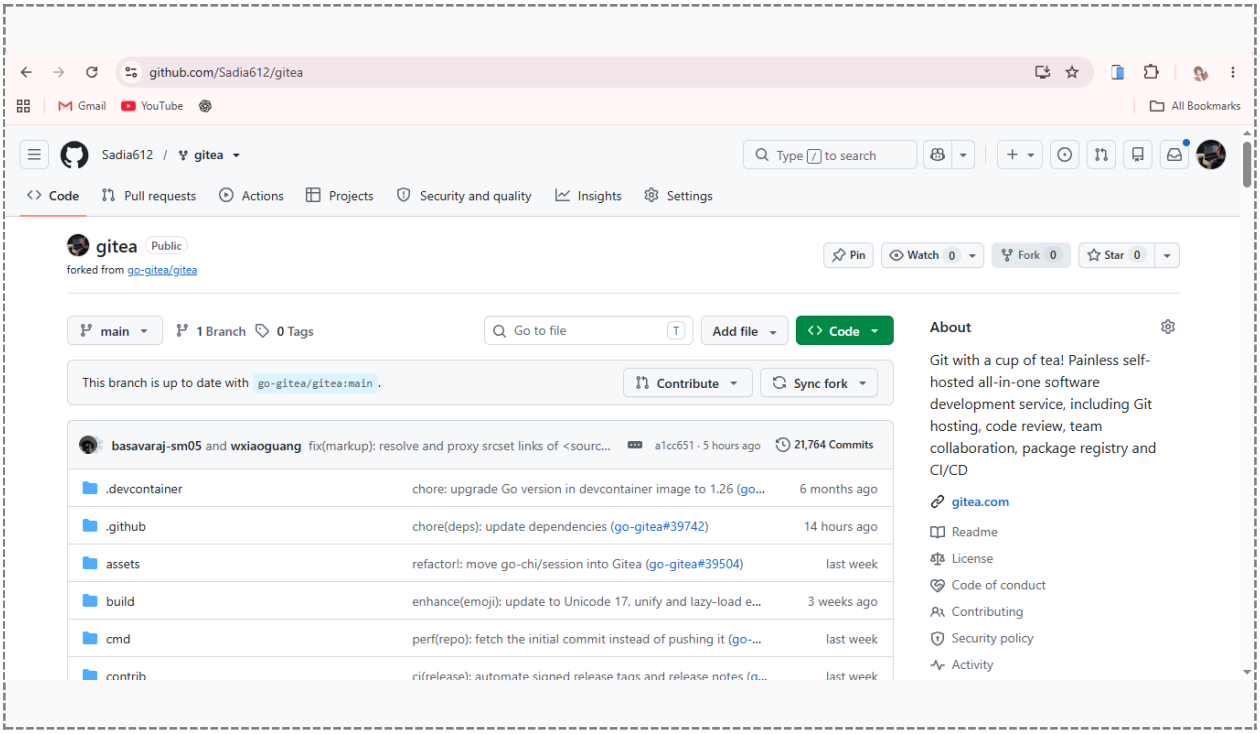


Figure: forked_gitea.png

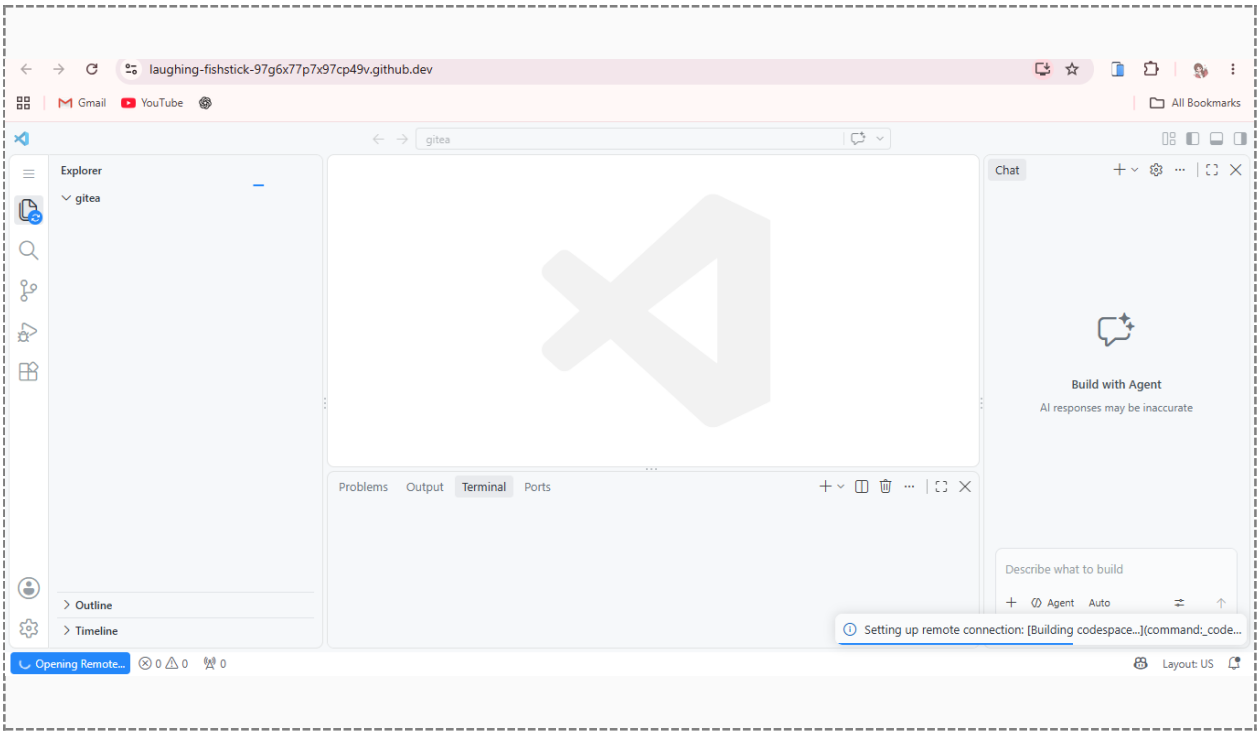


Figure: codespace_loading.png

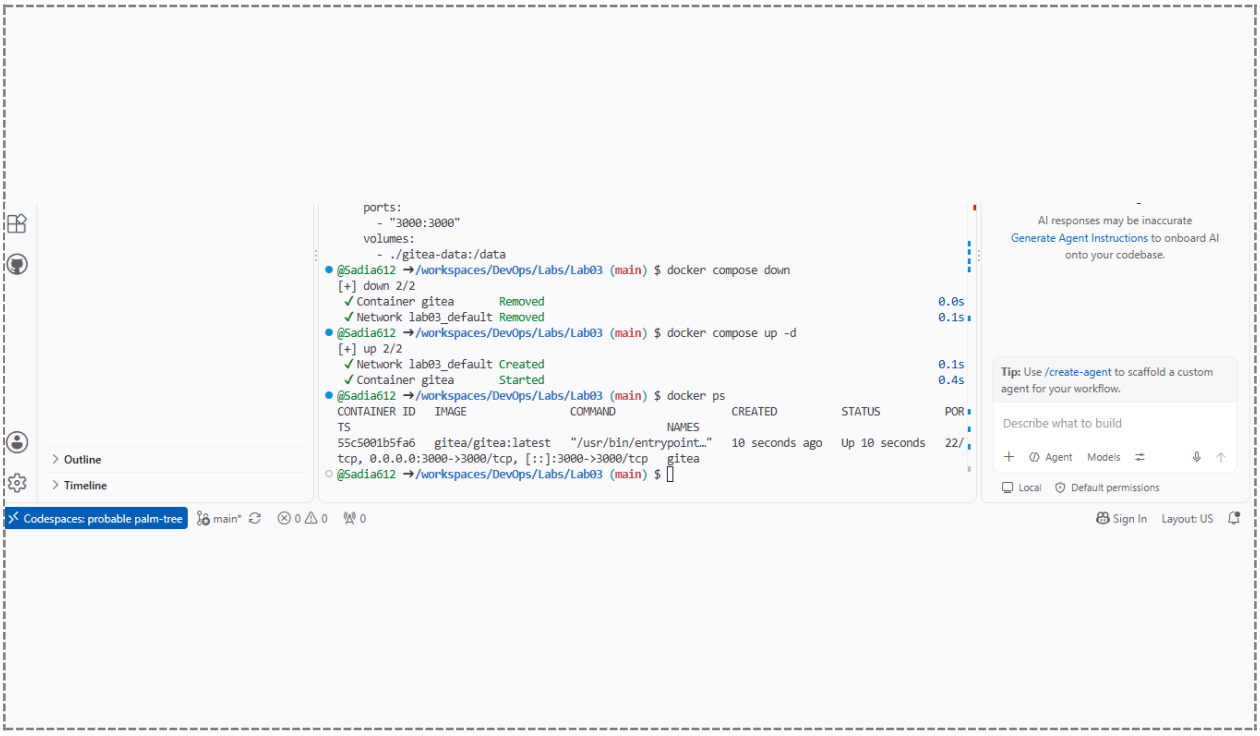


Figure: docker_up.png

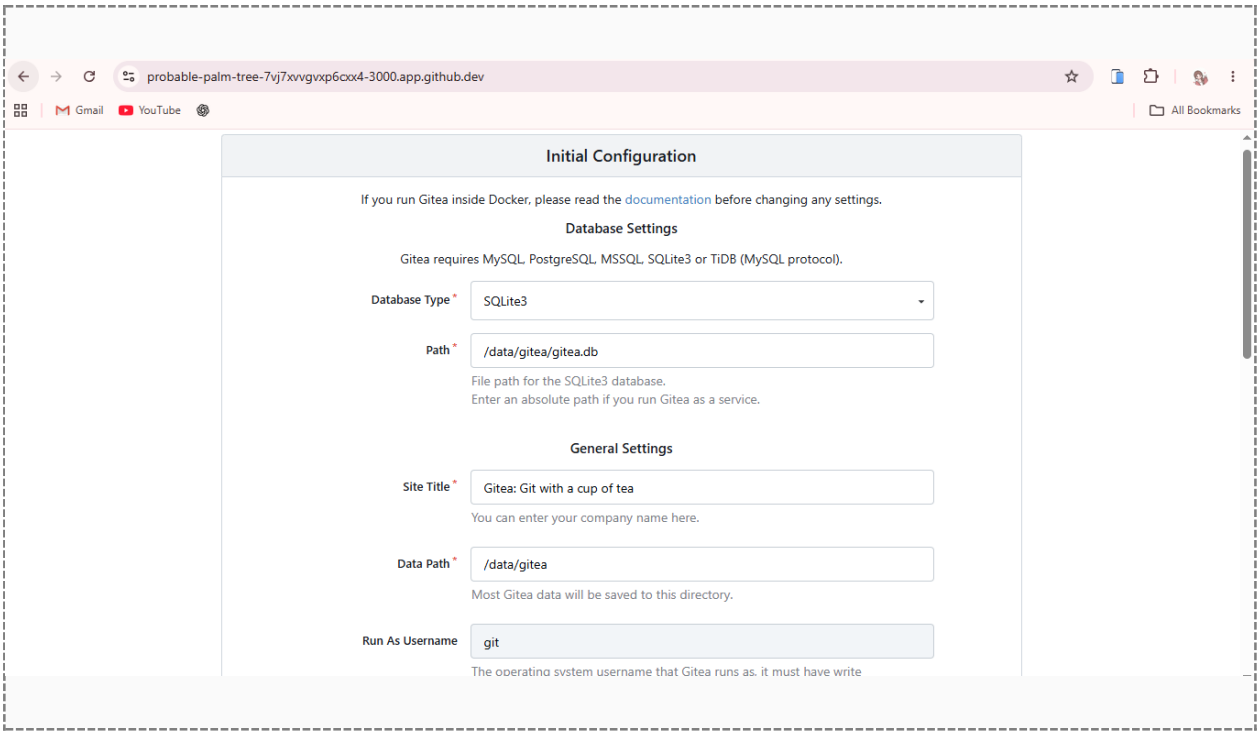


Figure: gitea_install_page.png

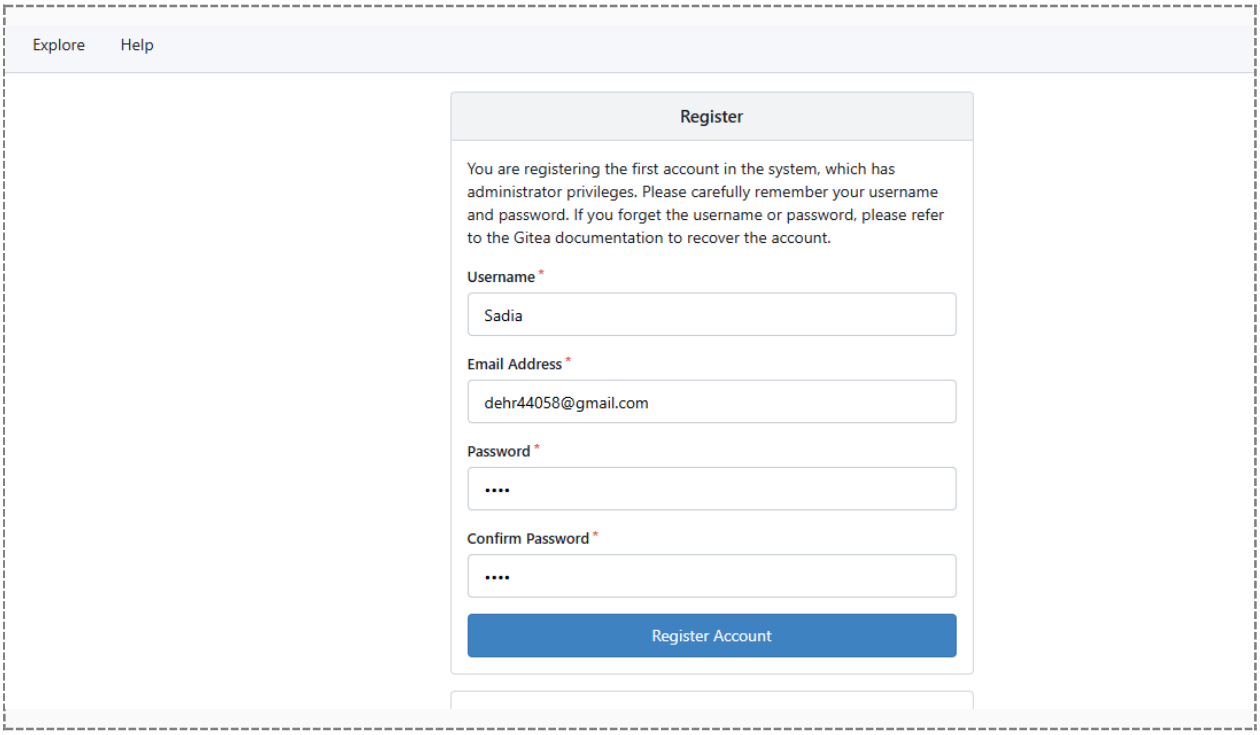


Figure: admin_setup.png

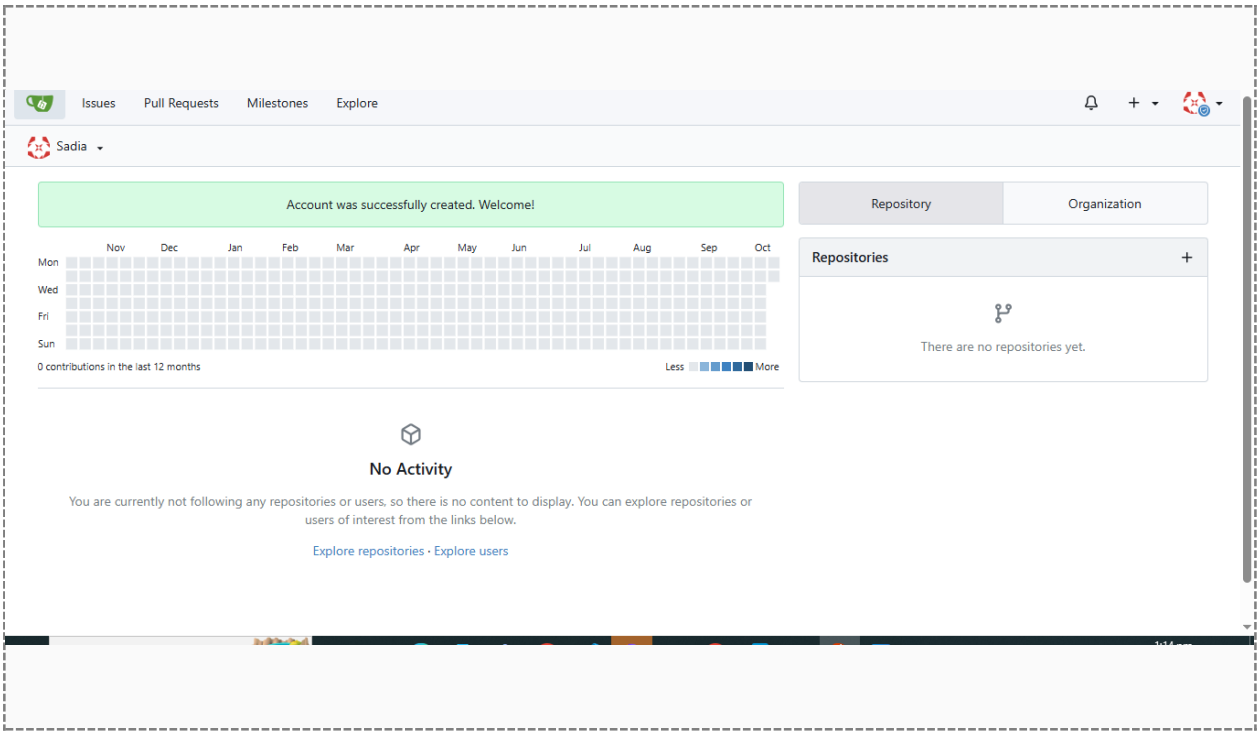


Figure: gitea_dashboard.png

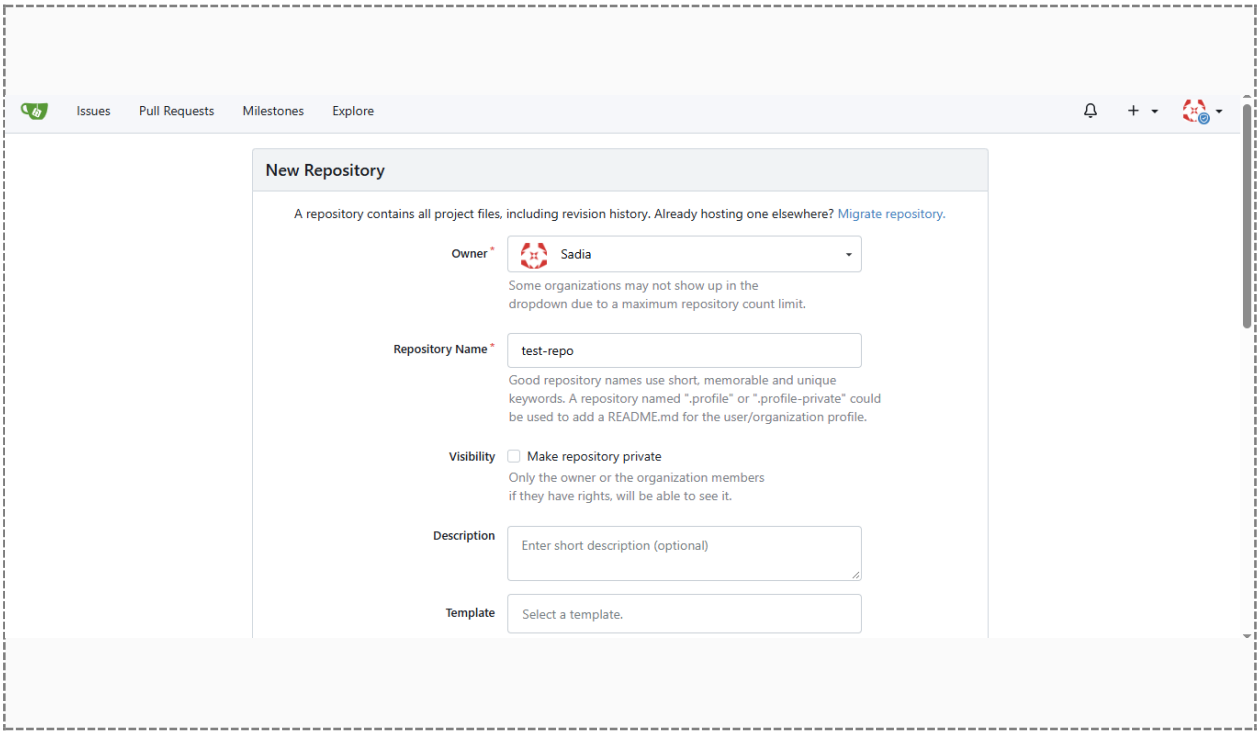


Figure: gitea_new_repo.png

Summary

A real application was run with Docker Compose in Codespaces, and a self-hosted Git service was installed and used.

Task 11 – Creating a GitHub Pages Portfolio Site

A repository named Sadia612.github.io was created, the portfolio files (index.html, profile.jpg) were added, GitHub Pages was enabled from the main branch, and the live site was verified.

Commands used

```
git init
git branch -M main
git remote add origin https://github.com/Sadia612/Sadia612.github.io.git
git add .
git commit -m "Add portfolio site for GitHub Pages"
git push -u origin main
```

Screenshots

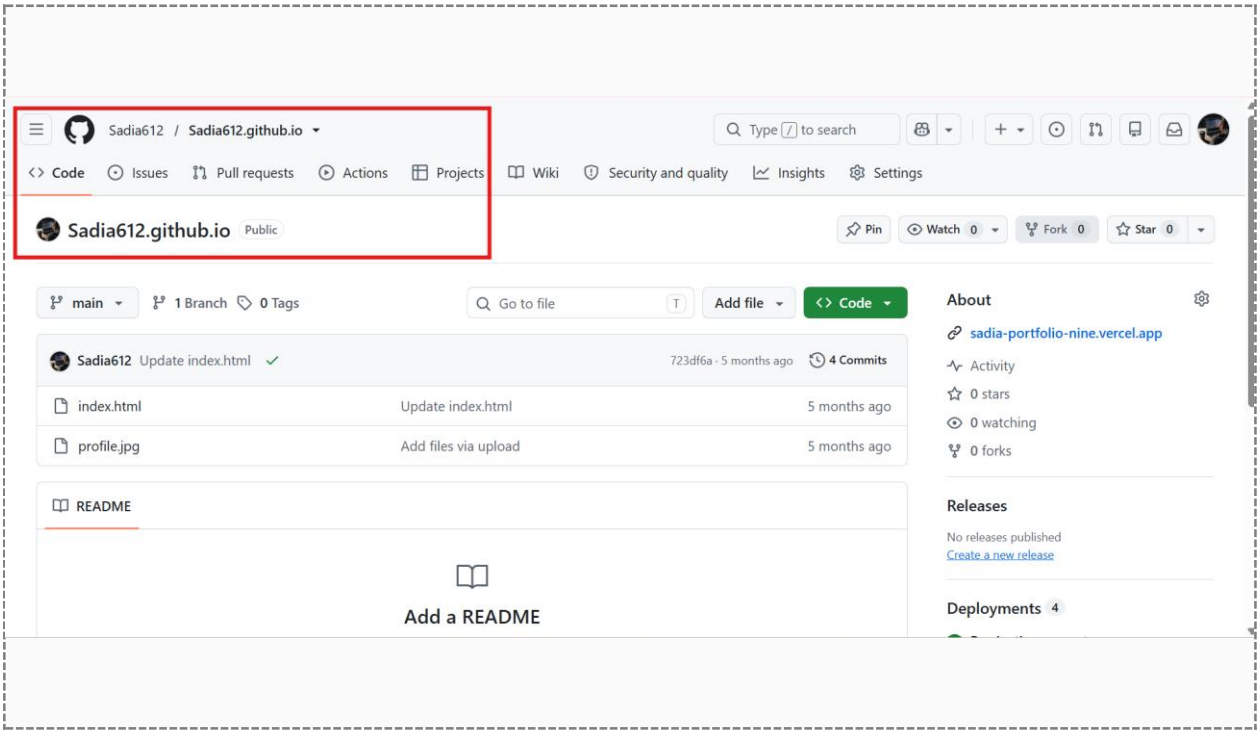


Figure: github_pages_repo.png

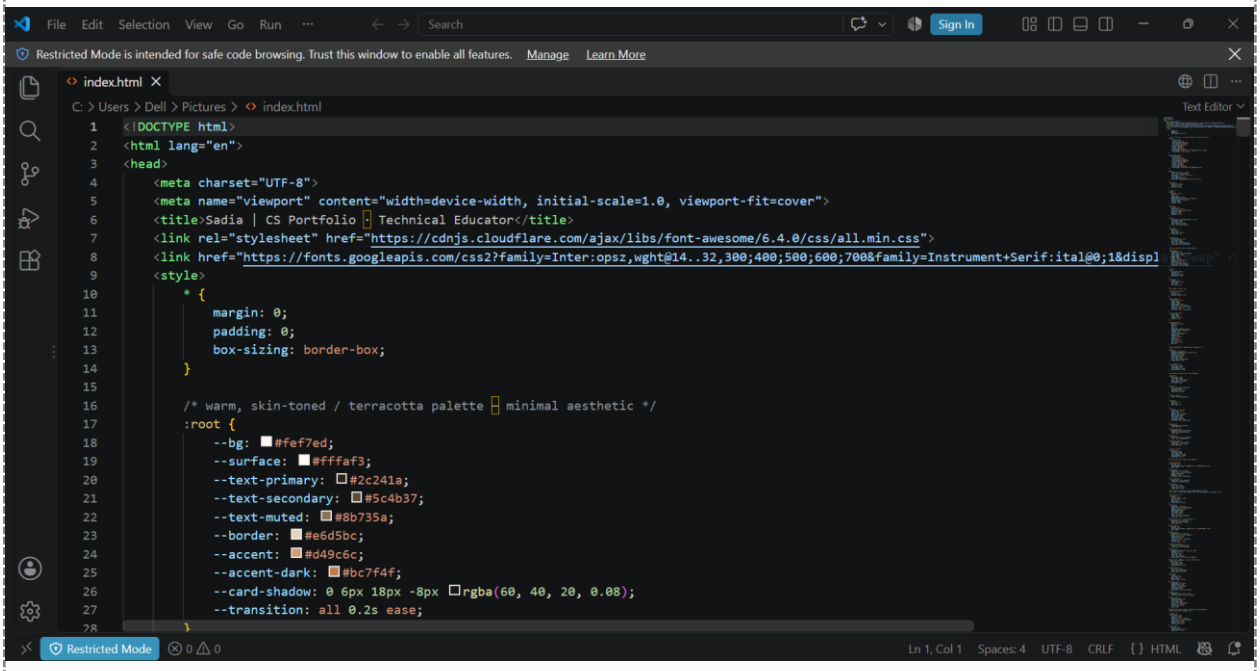


Figure: local_static_site.png

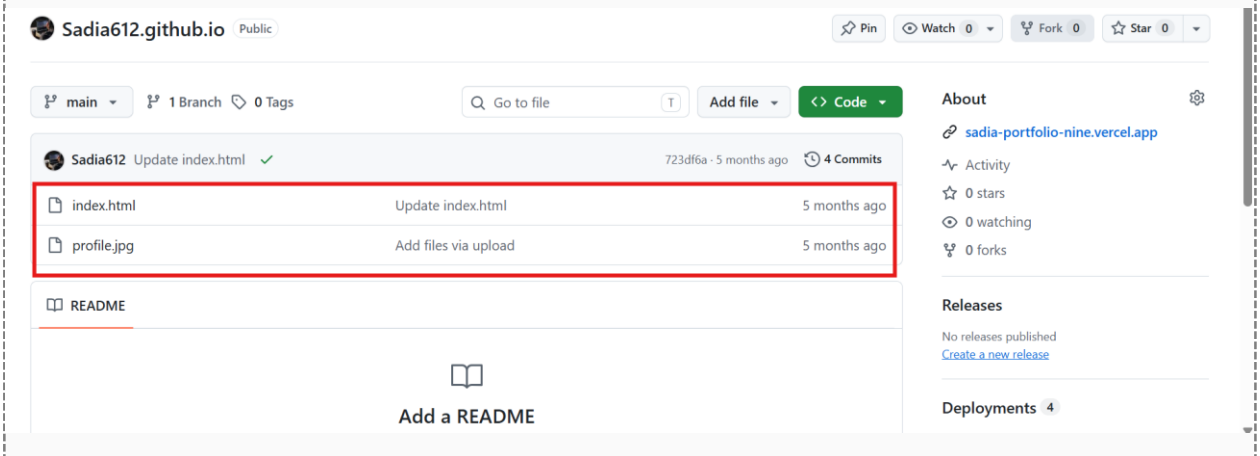


Figure: push_static_site.png

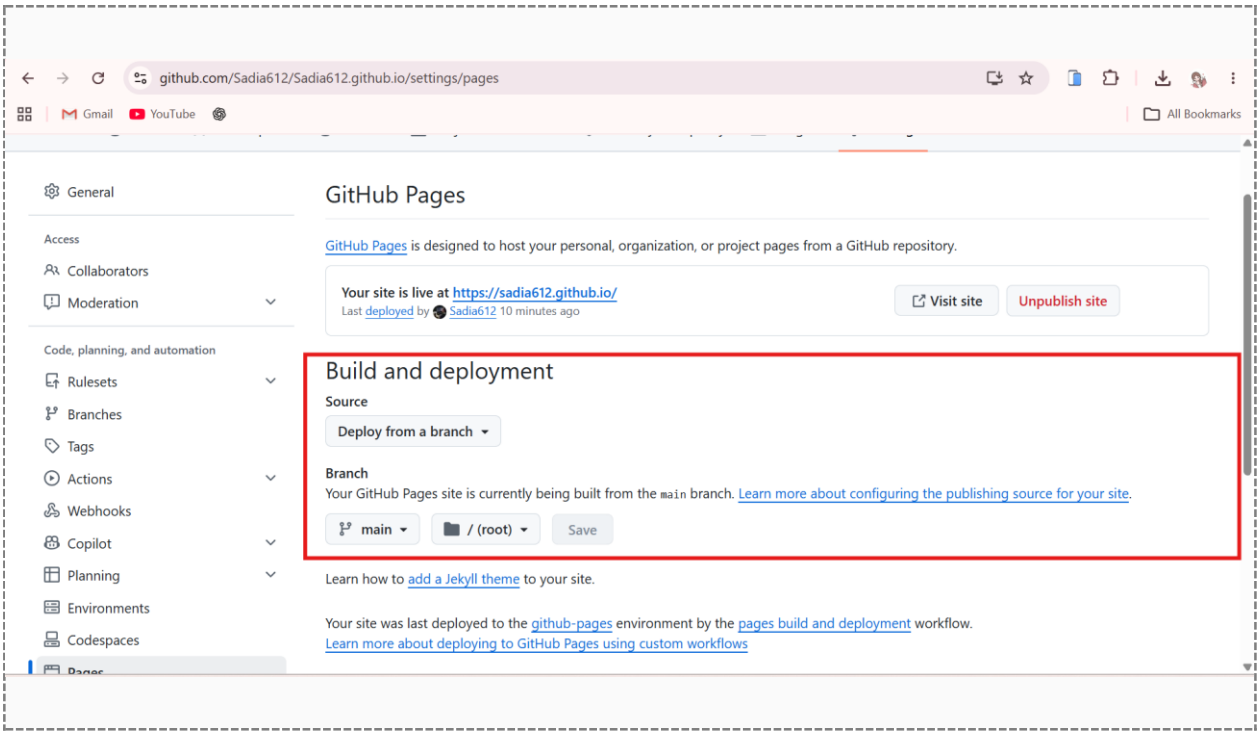


Figure: github_pages_settings.png

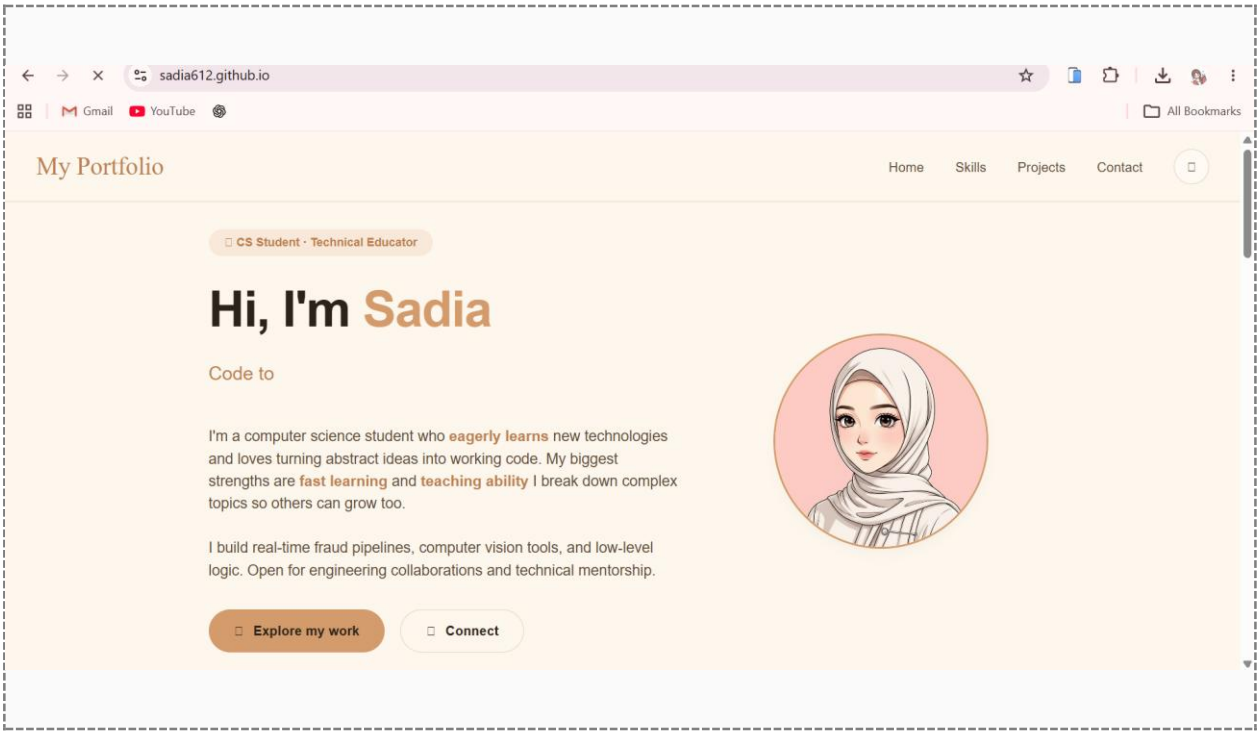


Figure: live_site.png

Summary

A GitHub Pages repository named after the username publishes a static site at https://sadia612.github.io.